

1. Принцип действия ЭВМ. Неймановские принципы построения компьютеров.

Пять классических компонентов любой ЭВМ:

- устройства ввода
- устройства вывода
- память
- путь данных (в лекции написано datapath я хз как еще перевести)
(тракт данных)
- устройство управления

Неймановские принципы:

- принцип хранимой в памяти программы
- двоичное кодирование
- программное управление (последовательное выполнение команд)
- однородность памяти (безразличие к целевому назначению данных)
Принстон
- линейное пространство памяти (адресность)

Принцип хранимой в памяти программы:

- ENIAC – (1946 -1955), программа задавалась схемой коммутации триггеров на 40 наборных полях.
- EDVAC – (1949-...) первая машина с хранимой в памяти программой

Принцип двоичного кодирования:

- Вся информация (команды, данные) – в бинарном виде
- Каждый тип информации имеет свой формат
- Последовательность битов, связанных одним смыслом, называется полем:
 - для данных – поле знака и поле значащих разрядов
 - для команды – поле кода операции и поле адресов

Принцип программного управления:

- Все вычисления алгоритма должны быть представлены в виде программы
- Программа состоит из последовательности управляющих слов - команд
- Команды хранятся в последовательных ячейках памяти и выполняются в естественной последовательности
- Последовательность может быть изменена – условно или безусловно

Принцип однородности памяти:

- Команды и данные хранятся в одной памяти и внешне не различимы
- Возможность производить над командами те же операции, что и над данными (модификация команд)
- Модификация команд современным программированием не приветствуется
- Самый полезный эффект данного принципа – команды одной программы могут быть получены как результат работы другой программы. Возможность **трансляции** – главное следствие
- Команды и данные хранятся в одной памяти и внешне не различимы – Принстонская архитектура
- Гарвардская архитектура – отдельная память команд и отдельная память данных.
- Узкое место принстонской архитектуры – пропускная способность тракта «процессор-память»
- В последнее время благодаря кэш-памяти всё чаще начинает использоваться гарвардская архитектура.

Принцип адресности:

- Память – набор пронумерованных ячеек
- В каждый момент доступна любая ячейка
- Двоичные коды данных и команд хранятся в единицах информации – словах
- Доступ к данным по номерам ячеек – адресам

2. Принстонская и Гарвардская архитектуры ЭВМ.

Архитектура фон Неймана (Принстонская)

Преимущества:

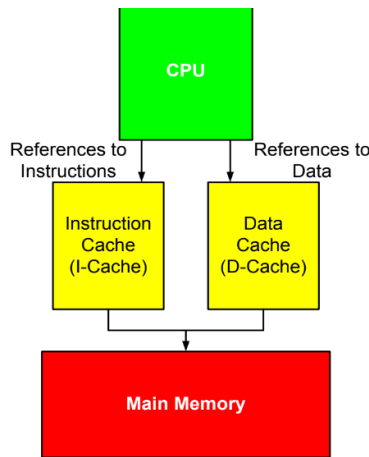
- Компьютеры с такой архитектурой легче проектировать и строить за счет простоты конструкции.
- Программы и данные хранятся в одной и той же памяти, что обеспечивает универсальность использования.
- При заданной емкости более вероятно попадание благодаря автоматическому балансу между командами и данными. Например, если в фрагменте программы больше команд, то и кэш будет насыщаться командами.

Недостатки:

- Все операции выполняются через один и тот же шинный канал, поэтому возможны узкие места и замедления, что приводит к ограниченной производительности.

- Могут возникнуть проблемы с безопасностью из-за того, что программы и данные хранятся в одной и той же памяти. Существует потенциальный риск перезаписи программ кодом.

Архитектура Гарварда (Гарвардская)



Преимущества:

- Раздельный кэш для программ и данных позволяют параллельно выполнять чтение/запись, что обеспечивает высокую производительность.
- Раздельный кэш для программ и данных позволяют параллельно выполнять чтение/запись. Это повышает производительность и эффективность доступа к памяти, поскольку процессор может одновременно обращаться к инструкциям и данным.
- За счет параллельного доступа к памяти программа и данные могут быть переданы процессору более эффективно, что приводит к увеличению пропускной способности системы (полоса пропускания).
- Большинство программ не изменяют собственные команды, т. к. самоизменяющиеся программы – слишком сложны и значительно повышают нагрузку на оба кэша.

Недостатки:

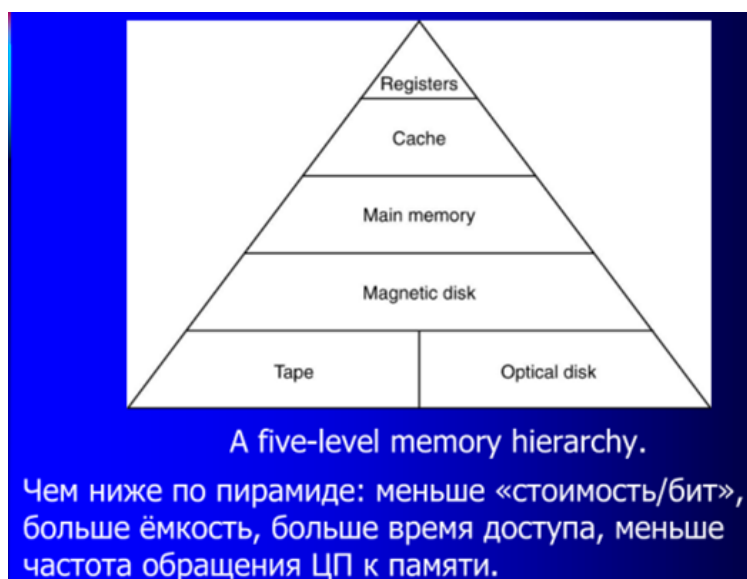
- Реализация требует более сложных средств управления памятью и адресации. Это может усложнить процесс проектирования и увеличить стоимость разработки.
- Наличие раздельного кэша для программ и данных может увеличивать стоимость и сложность конструкции процессора и памяти.

Многие современные компьютеры используют комбинированные архитектуры для достижения оптимального сочетания преимуществ. Например, процессоры могут иметь Гарвардскую кэш-память для улучшения производительности, сохраняя при этом основные черты архитектуры фон Неймана для упрощения программирования.

3. Современное состояние развития архитектур процессоров и ЭВМ. Обобщенная структура ЭВМ и ее функционирование. (Ефимчик)

4. Иерархия памяти ЭВМ. Классификация и основные параметры запоминающих устройств(ЗУ): внутренняя и внешняя память, кэш-ЗУ, статическая, динамическая и флэш-память ПК (персонального компьютера). (Давидович)

Память предназначена для фиксации, хранения и выдачи информации в процессе работы ЭВМ. Процессы чтения и записи информации определяются как процессы обращения к запоминающему устройству (ЗУ).



ЗУ характеризуются:

- Местом расположения (на кристалле ЦП, на плате, внешняя память).
- Ёмкостью - емкость ЗУ характеризуют числом битов либо байтов, которое может храниться в запоминающем устройстве. На практике применяются более крупные единицы – с приставками кило, мега, гига, тера, пета, экса (kilo, mega, giga, tera, peta, exa) – $10^3, 10^6, 10^9, 10^{12}, 10^{15}$ и 10^{18} . Ближе, но не равно $2^{10}, 2^{20}, 2^{30}, 2^{40}, 2^{50}$ и 2^{60} .
- Единицей пересылки - обычно равна ширине ШД (слову), но не обязательно. Из внешней памяти обычно данные передаются блоками.
- Методом доступа:
 - 1) Последовательный доступ. Хранение информации в виде последовательности

блоков данных, называемых записями. Для доступа к нужному элементу (слову или байту) необходимо прочитать все предшествующие ему данные. (Магнитные ленты)

2) Прямой доступ. Каждая запись имеет уникальный адрес, отражающий ее физическое размещение на носителе информации. Обращение - адресный доступ к началу записи и последующий последовательный доступ к единице информации внутри записи. (Жесткий диск)

3) Произвольный доступ. Каждая ячейка памяти имеет уникальный физический адрес. Обращение к любой ячейке занимает одно и то же время и может проводиться в произвольной очередности. (ОЗУ).

4) Ассоциативный доступ. Поиск ячеек, содержащих информацию, в которой значение отдельных битов совпадает с состоянием одноименных битов в заданном образце. Сравнение осуществляется параллельно для всех ячеек памяти, независимо от ее емкости. (КЭШ-память)

- Быстродействием:

Время доступа - для памяти с произвольным доступом оно соответствует интервалу времени от момента поступления адреса до момента, когда данные заносятся в память или становятся доступными. В ЗУ с подвижным носителем информации это время, затрачиваемое на установку головки записи/считывания (или носителя) в нужную позицию.

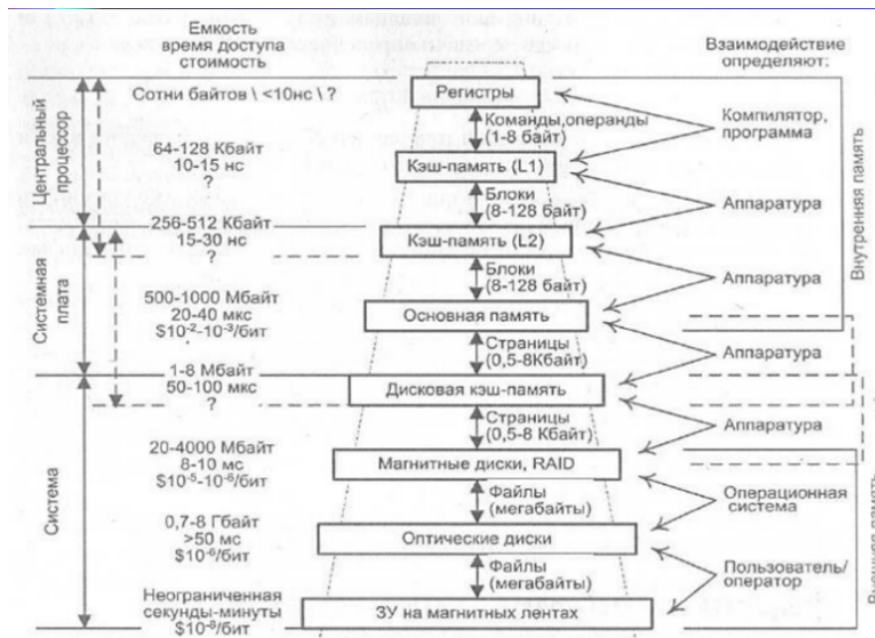
Длительность цикла памяти или период обращения (ТЦ). Понятие применяется к памяти с произвольным доступом, для которой оно означает минимальное время между двумя последовательными обращениями к памяти. Период обращения включает в себя время доступа плюс некоторое дополнительное время. Дополнительное время может требоваться для затухания сигналов на линиях, а в некоторых типах ЗУ, где считывание информации приводит к ее разрушению, для восстановления считанной информации.

Скорость передачи. Это скорость, с которой данные могут передаваться в память или из нее. Для памяти с произвольным доступом она равна $1/T_C$. Для других видов памяти скорость передачи определяется соотношением:

$T_N = T_A + N/R$, где T_N - среднее время считывания или записи N битов; T_A – среднее время доступа; R - скорость пересылки в битах в секунду.

Стоимость – отношение общей стоимости ЗУ к его ёмкости в битах
 -> стоимость хранения одного бита информации.

- Физическим типом (полупроводники, магнитный носитель, оптика)
- Физические особенности (энерго – зависимая/независимая)



Основная память (ОП) -> единственный вид памяти, к которой ЦП может обращаться непосредственно.

Основная память - ЗУ с произвольным доступом. Основная память может включать в себя два типа устройств:

- оперативные запоминающие устройства (ОЗУ)
- постоянные запоминающие устройства (ПЗУ).

ОЗУ – RAM (Random Access Memory). Для большинства типов полупроводниковых ОЗУ характерна энергозависимость: даже при кратковременном прерывании питания хранимая информация теряется. Микросхема ОЗУ должна быть постоянно подключена к источнику питания и поэтому может использоваться только как временная память.

Вторую группу полупроводниковых ЗУ основной памяти образуют энергонезависимые микросхемы ПЗУ (ROM - Read-Only Memory). ПЗУ обеспечивает считывание информации, но не допускает ее изменения (в ряде случаев информация в ПЗУ может быть изменена,

но этот процесс сильно отличается от считывания и требует значительно большего времени).

Энергозависимые ОЗУ можно подразделить на две основные подгруппы:

- динамическую память (DRAM - Dynamic Random Access Memory)
- статическую память (SRAM - Static Random Access Memory).

В **статических** ОЗУ запоминающий элемент может хранить записанную информацию неограниченно долго (при наличии питающего напряжения).

Запоминающий элемент **динамического** ОЗУ способен хранить информацию только в течение достаточно короткого промежутка времени, после которого информацию нужно восстанавливать заново, иначе она будет потеряна. Динамические ЗУ, как и статические, энергозависимы.

Роль запоминающего элемента в статическом ОЗУ выполняет триггер (схема с двумя устойчивыми состояниями, обычно состоящую из четырех или шести транзисторов).

FLASH - Флэш-память появилась в конце 1980-х годов (Intel) – перепрограммируемая ПЗУ с электрическим стиранием. Стирание осуществляется сразу целой области ячеек : блока или всей микросхемы -> более быстрая запись информации (программирование ЗУ).

Для упрощения записи – спец. блоки ИМС, делающие запись “прозрачной ” для ПО и внешних устройств.

Флэш-память строится на одностранзисторных элементах памяти (с “плавающим ” затвором), -> плотность хранения информации чуть выше, чем DRAM.

Различные технологии построения базовых элементов флэш - количество слоев, методы стирания и записи данных, структура. Наиболее широко известны NOR и NAND типы флэш-памяти , запоминающие транзисторы в которых подключены к разрядным шинам - параллельно и последовательно.

Минусы – относительно невысокая скорость передачи данных.

Элементы памяти флэш -ЗУ организованы в матрицы, разрядность данных для микросхем - 1-2 байта .

Операция чтения из флэш - как в обычных ЗУ с произвольным доступом (ОЗУ или кэш). Но запись – с некоторыми особенностями, аналогичными - ПЗУ. Перед записью данных в ячейки они должны быть очищены (стерты). Стирание - перевод элементов памяти в состояние единицы и возможно только сразу для целого блока ячеек (в первых микросхемах предусматривалось стирание только для всей матрицы сразу- Bulk Erase). Выборочное - невозможно.

В процессе записи информации соответствующие элементы памяти переключаются в нулевое состояние. Также , как и в ПЗУ , без стирания можно дозаписать нули в уже запрограммированные ячейки, однако необходимость в такой операции относительно редка.

Простой кэш

- Полностью ассоциативный: любой блок данных ОЗУ может быть помещён в любое место кэш
- LRU (least recently used) стратегия замещения: при недостатке места выкидывается наиболее давно требовавшиеся данные.

В кэш с прямым отображением (direct mapped cache), каждому блоку памяти поставлена в соответствие отдельная строка кэш-памяти.

- Прямое отображение (Direct-Mapped)

– Требуется меньше места

- Только один компаратор

- Требуется меньше битов для тэгов

– Быстрый: может возвращать данные процессору параллельно с определением попадания или промаха

– Поочерёдное обращение к словам из разных блоков, размещаемых в одной строке кэш (Conflict misses) – снижение вероятности попадания

- Полностью ассоциативное отображение (Fully-Associative)

– Нет конфликтов (conflict misses) → общая высокая вероятность попадания

– Требуется дорогая ассоциативная память – отдельный компаратор для каждой строки кэш

- Direct mapped кэш проще:

– Меньше аппаратуры; более быстр в потенциале

- Fully associative - меньше промахов и конфликтов.

- Кэш, совмещающий преимущества обоих способов:

– Индекс вычисляется из адреса

– В “k-way set associative cache” индекс определяет множество k модулей кэш, где могут храниться данные (т.е. k вариантов размещения для каждого блока данных ОЗУ).

- k=1 – прямое отображение (direct mapped).

- k= размеру кэша (в строках) – полностью ассоциативное.

– Используется замещение LRU (или другое) для всего набора.

Важным звеном в иерархии запоминающих устройств является **внешняя**, или вторичная память, реализуемая на базе различных ЗУ. Наиболее распространенные виды таких ЗУ — это магнитные и оптические диски и магнитоленточные устройства.

Внешняя

1) Магнитные диски

Информация в ЗУ на магнитных дисках (МД) хранится на плоских металлических или пластиковых пластинах (дисках), покрытых магнитным материалом. Данные записываются и считываются с диска с помощью электромагнитной катушки», называемой головкой считывания/записи, которая в процессе считывания и записи неподвижна, в то время как диск вращается относительно нее. При записи на головку подаются электрические импульсы, намагничивающие участок поверхности под ней, причем характер намагниченности поверхности различен в зависимости от направления тока в катушке. Считывание базируется на электрическом токе, наводимом в катушке головки, под воздействием перемещающегося относительно нее магнитного поля. Когда под головкой проходит участок поверхности диска, в катушке наводится ток той же полярности, что использовался для записи

информации. Несмотря на разнообразие типов магнитных дисков, принципы их организации обычно однотипны.

Данные на диске организованы в виде набора концентрических окружностей, называемых дорожками. Каждая из них имеет ту же ширину, что и головка. Соседние дорожки разделены промежутками. Это предотвращает ошибки из-за смещения головки или из-за интерференции магнитных полей. Как правило, для упрощения электроники принимается, что всех дорожках может храниться одинаковое количество. Таким образом, плотность записи увеличивается от внешних дорожек к внутренним.

Обмен информацией с МД осуществляется блоками. Размер блока обычно меньше емкости дорожки, и данные на дорожке хранятся в виде последовательных областей — секторов, разделенных между собой промежутками. Размер сектора равен минимальному размеру блока.

Типовое число секторов на дорожке колеблется от 10 до 100. При такой организации должны быть заданы точка отсчета секторов и способ определения начала и конца каждого сектора. Все это обеспечивается с помощью форматирования, в ходе которого на диск заносится служебная информация, недоступная пользователю и используемая только аппаратурой дискового ЗУ.

2) Магнитные ленты

ЗУ на базе магнитных лент используются в основном для архивирования информации. Носителем служит тонкая полистироловая лента шириной от 0,38-2,54 см и толщиной около 0,025 мм, покрытая магнитным слоем. Лента наматывается на бобины различного диаметра. Данные записываются последовательно, байт за байтом, от начала ленты до ее конца. Время доступа к информации на магнитной ленте значительно больше, чем у ранее рассмотренных видов внешней памяти.

Обычно вдоль ленты располагается 9 дорожек, что позволяет записывать поперек ленты байт данных и бит паритета. Информация на ленте группируется в блоки — записи. Каждая запись отделяется от соседней межблочным промежутком, дающим возможность позиционирования головки считывания/записи на начало любого блока. Идентификация записи производится по полю заголовка, содержащемуся в каждой записи. Для указания начала и конца ленты используются физические маркеры в виде металлизированных

полосок, наклеиваемых на магнитную ленту, или прозрачных участков на самой ленте. Известны также варианты маркирования начала и конца ленты путем записи на нее специальных кодов-индикаторов.

В универсальных ВМ обычно применяются бобинные устройства с вакуумными системами стабилизации скорости перемещения ленты. В них скорость перемещения ленты составляет около 300 см/с, плотность записи — 4 Кбайт/см, а скорость передачи информации - 320 Кбайт/с. Типовая бобина содержит 730 м магнитной ленты.

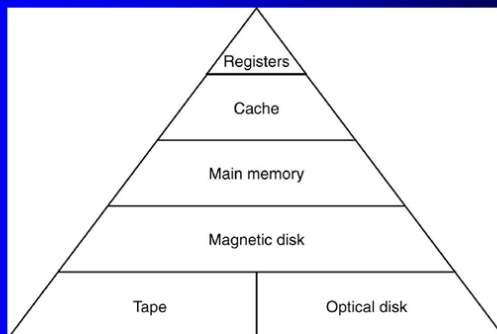
В ЗУ на базе картриджей используются кассеты с двумя катушками, аналогичные стандартным аудиокассетам. Типовая ширина ленты — 8 мм. Наиболее распространенной формой таких ЗУ является DAT (Digital Audio Tape). Данные на ленту заносятся по диагонали, как это принято в видеокассетах. По размеру такой картридж примерно вдвое меньше, чем обычная компакт-кассета, и имеет толщину 3,81 мм. Каждый картридж позволяет хранить несколько гигабайтов данных. Время доступа к данным невелико (среднее между временами доступа к дискетам и к жестким дискам). Скорость передачи информации выше, чем у дискет, но ниже, чем у жестких дисков.

Вторым видом ЗУ на базе картриджей является устройство стандарта DDS (Digital Data Storage). Этот стандарт был разработан в 1989 году для удовлетворения требований к резервному копированию информации с жестких дисков в мощных серверах и многопользовательских системах. В сущности, это вариант DAT, обеспечивающий хранение 2 Гбайт данных при длине ленты 90 м. В более позднем варианте стандарта DDS-DC (Digital Data Storage - Data Compression) за счет применения методов сжатия информации емкость ленты увеличена до 8 Гбайт.

Наконец, третий вид ЗУ на базе картриджей также предназначен для резервного копирования содержимого жестких дисков, но при меньших объемах такой информации. Этот тип ЗУ отвечает стандарту QIC (Quarter Inch Cartridge tape) и более известен под названием стример. Известны стримеры, обеспечивающие хранение от 15 до 525 Мбайт информации. В зависимости от информационной емкости и фирмы-изготовителя изменяются и характеристики таких картриджей. Так, число дорожек может варьироваться в диапазоне от 4 до 28, длина ленты — от 36 до 300 м и т. д.

5. Место оперативной памяти в структуре ЭВМ. Адресные запоминающие устройства. (Мешков)

Memory Hierarchies



A five-level memory hierarchy.

Чем ниже по пирамиде: меньше «стоимость/бит», больше ёмкость, больше время доступа, меньше частота обращения ЦП к памяти.

Слайд 14

Основная память (ОП) -> единственный вид памяти, к которой ЦП может обращаться непосредственно. Основная память - ЗУ с произвольным доступом.

Основная память может включать в себя два типа устройств:

- оперативные запоминающие устройства (ОЗУ) и
- постоянные запоминающие устройства (ПЗУ).

ОЗУ – RAM (Random Access Memory). Для большинства типов полупроводниковых ОЗУ характерна энергозависимость: *даже при кратковременном прерывании питания хранимая информация теряется. Микросхема ОЗУ должна быть постоянно подключена к источнику питания и поэтому может использоваться только как временная память.*

Слайд 21

В **статических ОЗУ** запоминающий элемент может хранить записанную информацию неограниченно долго (при наличии питающего напряжения).

Запоминающий элемент **динамического ОЗУ** способен хранить информацию только в течение достаточно короткого промежутка времени, после которого информацию нужно восстанавливать заново, иначе она будет потеряна. Динамические ЗУ, как и статические, энергозависимы.

Роль запоминающего элемента в статическом ОЗУ выполняет триггер (схема с двумя устойчивыми состояниями, обычно состоящую из четырех или шести транзисторов).

Слайд 23

Адресные ЗУ представлены в классификации статическими и динамическими оперативными устройствами и памятью типа ROM. Многочисленные варианты этих ЗУ имеют много общего с точки зрения структурных схем, что делает более рациональным изучение некоторых обобщенных структур с последующим описанием запоминающих элементов для различных ЗУ.

Общность структур особенно проявляется для статических ОЗУ и памяти типа ROM. Структуры динамических ОЗУ имеют свою специфику. Для статических ОЗУ и памяти типа ROM наиболее характерны структуры 2D, 3D и 2DM.

В структуре 2D запоминающие элементы организованы в прямоугольную матрицу размерностью $M = k \cdot m$, где M – информационная емкость памяти в битах; k – число хранимых слов (число строк матрицы); m – их разрядность (число колонок матрицы).

Дешифратор адресного кода при наличии разрешающего сигнала активизирует одну из выходных линий, разрешая одновременный доступ ко всем элементам выбранной строки, хранящей слово, адрес которого соответствует номеру строки. Элементы одного столбца соединены вертикальной линией – внутренней линией данных (разрядной линией). Элементы столбца хранят одноименные биты всех слов. Направление обмена (чтение или запись) определяется уровнем сигнала.

Структура типа 2D применяется лишь в ЗУ малой информационной емкости, так как при росте емкости проявляется несколько ее недостатков, наиболее очевидным из которых является чрезмерное усложнение дешифратора адреса (число выходов дешифратора равно числу хранимых слов).

Структура 3D позволяет резко упростить дешифраторы адреса с помощью двухкоординатной выборки запоминающих элементов. Принцип двухкоординатной выборки поясняется на примере ЗУ типа ROM, реализующего только операцию чтения данных (рисунок 18, а).

Код адреса разрядностью n делится на две половины, каждая из которых декодируется отдельно. Выбирается запоминающий элемент, находящийся на пересечении активных линий выходов обоих дешифраторов, т.е. на пересечении выбранной строки и колонки. Таких пересечений будет 2^n , следовательно, матрица хранит 2^n одноразрядных слов.

Уже для ЗУ небольшой емкости видно, что для структуры 2D при хранении 1 К слов потребовался бы дешифратор с 1 024 выходами, тогда как для структуры типа 3D нужны два дешифратора с 32 выходами каждый. Недостатком структуры 3D в первую очередь является усложнение запоминающих элементов, имеющих двухкоординатную выборку.

Структура типа 3D, показанная на рисунке 18, а с одноразрядной организацией, может применяться и в ЗУ с многоразрядной организацией (рисунок 18, б). В этом случае несколько матриц управляются от двух дешифраторов, относительно которых они включены параллельно. Каждая матрица выдает один бит адресованного слова, а число матриц равно разрядности хранимых слов. Структуры типа 3D имеют также довольно

ограниченное применение, поскольку в структурах типа 2DM (2D модифицированная) сочетаются достоинства обеих рассмотренных структур: упрощается дешифрация адреса и не требуются запоминающие элементы с двухкоординатной выборкой.

3У тина ROM структуры 2DM (рисунок 19) для матрицы запоминающих элементов с адресацией от дешифратора DC_x имеет как бы характер структуры 2D: активный выход дешифратора выбирает целую строку. Однако, в отличие от структуры 2D, длина строки не равна разрядности хранимых слов, а многократно ее превышает. При этом число строк матрицы уменьшается и, соответственно, уменьшается число выходов дешифратора. Для выбора одной из строк служат не все разряды адресного кода, а их часть $A_{n-1} \dots A_k$. Остальные разряды адреса (от A_{k-1} до A_0) используются, чтобы выбрать необходимое слово из того множества слов, которое содержится в строке. Это выполняется с помощью мультиплексоров, на адресные входы которых подаются разряды $A_{k-1} \dots A_0$. Длина строки равна $m2^k$, где m – разрядность хранимых слов, а k – число разрядов адреса A_2 . Из каждого «отрезка» строки длиной 2^k мультиплексор выбирает один бит, на выходах мультиплексоров формируется выходное слово. По разрешению сигнала, поступающего на входы управляемых буферов с тремя состояниями, выходное слово передается на внешнюю шину.

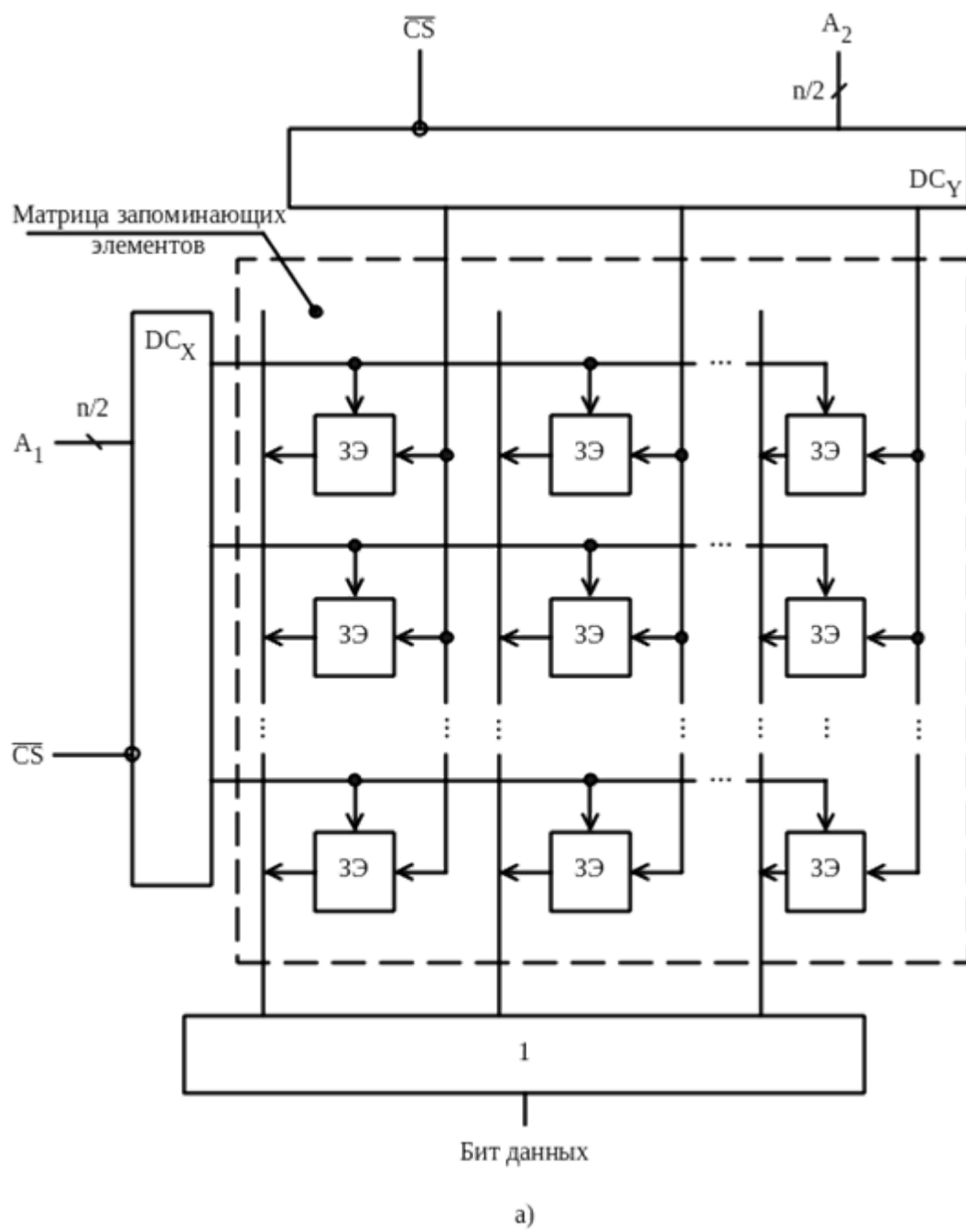
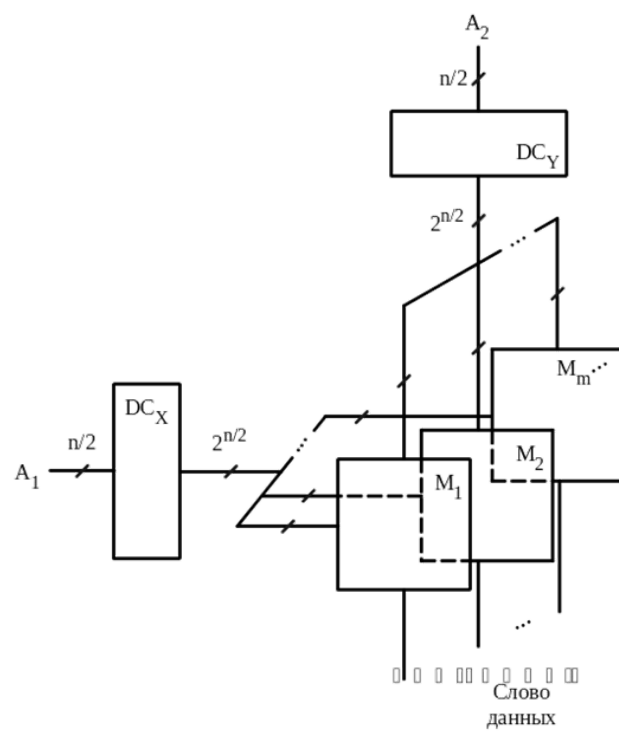


Рисунок 18 – Структура постоянного ЗУ типа 3D с одноразрядной (а) и многоразрядной организациями (б)



б)

Окончание рисунка 18

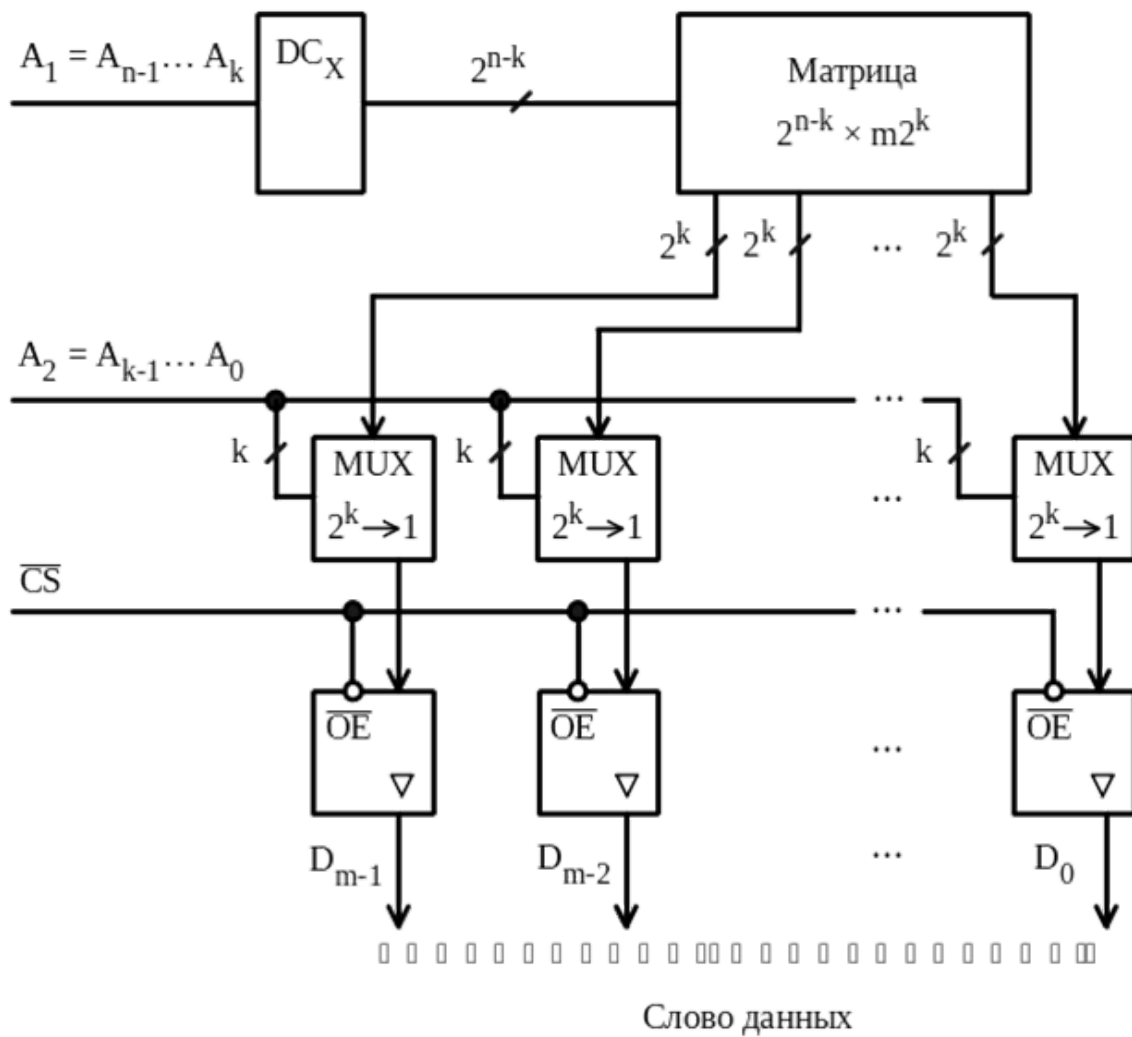


Рисунок 19 – Структура 3У типа 2DM для ROM

На рисунке 20 в более общем виде структура 2DM показана для 3У типа RAM с операциями чтения и записи. Из матрицы по-прежнему считывается «длинная» строка.

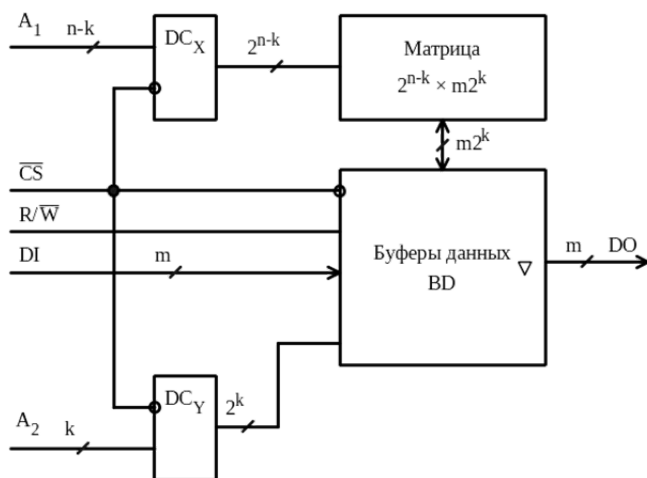


Рисунок 20 – Структура 3У типа 2DM для RAM

Данные в нужный отрезок этой строки записываются (или считываются из нее) управляемыми буферами данных BD, воспринимающими выходные сигналы второго дешифратора DC_Y и выполняющими не только функции мультиплексирования, но и функции изменения направления передачи данных под воздействием сигнала .

Кэш-память

Кэш-память запоминает копии информации, передаваемой между устройствами (прежде всего между процессором и основной памятью). Она имеет небольшую емкость в сравнении с основной памятью и более высокое быстродействие (реализуется на триггерных элементах памяти).

При чтении данных сначала выполняется обращение к Кэш-памяти (рисунок 21). Если в КЭШе имеется копия данных, адресованной ячейки основной памяти, то КЭШ вырабатывает сигнал **Hit** (попадание) и выдает данные на общую шину данных. В противном случае сигнал Hit не вырабатывается, и выполняется чтение из основной памяти и одновременное помещение считанных данных в КЭШ.

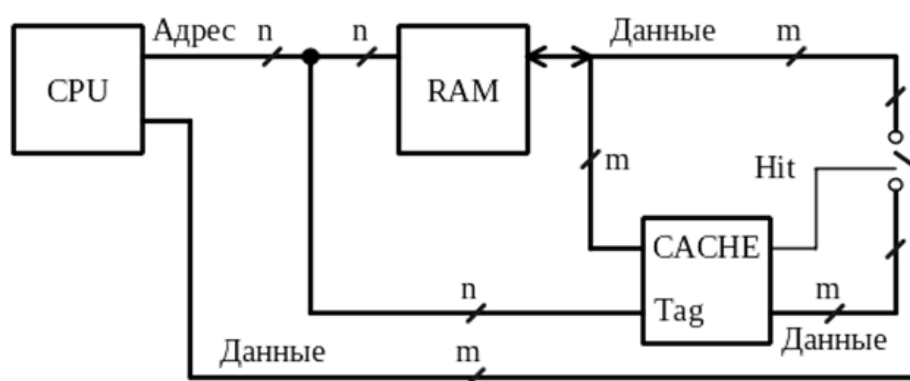


Рисунок 21 – Структура Кэш-памяти

Эффективность кэширования обуславливается тем, что большинство прикладных программ имеют циклический характер и многократно используют одни и те же данные. Поэтому после первого использования данных из относительно медленной основной памяти повторные обращения требуют меньше времени. К тому же при использовании процессором Кэш-памяти основная память освобождается, и могут выполняться регенерация данных в динамическом ЗУ или использование памяти другими устройствами.

Объем Кэш-памяти много меньше емкости основной памяти, и любая единица информации, помещаемая в КЭШ, должна сопровождаться дополнительными данными (тегом), определяющими копией содержания, какой ячейки основной памяти является эта единица информации.

6. Постоянные перезаписываемые ЗУ (ППЗУ), типа EPROM (Electrically Erasable Programmable ReadOnly Memory). Основные структуры ППЗУ. Многобанковая организация и расслоение ЗУ.(Ледохович)

Многократно программируемые ПЗУ. Процедура программирования таких ПЗУ имеет два этапа:

- стирание содержимого всех или части ячеек,
- запись новой информации.

В этом классе ПЗУ выделяют:

- EPROM (Erasable Programmable ROM - стираемые программируемые ПЗУ);
- EEPROM (Electrically Erasable Programmable ROM - электрически стираемые программируемые ПЗУ);
- флэш-память.

EPROM. Запись информации производится электрическими сигналами, так же как в PROM (однократно перепрограммируемое ПЗУ), однако перед операцией записи содержимое всех ячеек должно быть приведено к одинаковому состоянию (стерто) путем воздействия на микросхему ультрафиолетовым облучением. Процесс стирания может выполняться многократно. Каждое стирание занимает 20 мин.

- Данные хранятся в виде зарядов плавающих затворов МОП-транзисторов.
- Программирование микросхемы происходит с использованием технологии инжекции горячих электронов.

EEPROM. Однако, в отличие от EPROM, EEPROM позволяет перезаписывать данные в отдельных ячейках памяти, без необходимости стирать всю память целиком.

- Для записи и стирания данных в ячейке EEPROM используется туннельный эффект. Это достигается созданием достаточно сильного электрического поля в тонком диэлектрике между каналом транзистора и карманом (зарядовой областью). При наличии этого поля происходит туннелирование электронов, что позволяет изменять заряд в ячейке.
- В процессе записи применяется ускорение электронов, что усиливает туннелирование. Это достигается пропусканием тока через канал полевого транзистора.
- Чтение данных выполняется полевым транзистором, где карман выполняет роль затвора. Потенциал плавающего затвора изменяет пороговые характеристики транзистора, что может быть обнаружено цепями чтения.

Основные структуры:

Микросхемы ПЗУ построены по принципу *матричной структуры* накопителя, где в узлах расположены *перемычки* (проводники, полупроводниковые диоды или транзисторы) подключенные к адресной линии и разрядной линии считывания. Наличие перемычки может означать 1, а ее отсутствие — 0.

Многобанковая структура и расслоение памяти.

Увеличение разрядности ЗУ реализуется за счет объединения адресных входов объединяемых ИМС ЗУ. Информационные входы и выходы микросхем являются входами и выходами модуля ЗУ увеличенной разрядности. Полученную совокупность микросхем называют *модулем памяти*.

Один или несколько модулей образуют *банк памяти*.

Для получения требуемой емкости ЗУ нужно объединить несколько банков памяти меньшей емкости. В общем случае основная память ВМ практически всегда имеет *блочную структуру*, то есть содержит несколько банков.

При использовании блочной памяти, состоящей из В банков, адрес ячейки А преобразуется в пару (Ъ, w) где Ъ - номер банка, w - адрес ячейки внутри банка.

Схемы распределения разрядов адреса А между Ъ и w:

- блочная (номер банка Ъ определяет старшие разряды адреса);
- циклическая ($\text{Ъ} = A \text{ div } B$);
- блочно-циклическая (комбинация двух предыдущих схем).

Блочное построение памяти позволяет сократить время доступа к информации. Это возможно благодаря потенциальному *параллелизму*. Большой скорости доступа можно достичь за счет одновременного доступа ко многим банкам памяти. Одна из используемых для этого методик - *расслоение памяти*.

В её основе лежит так называемое *чередование адресов* (address interleaving), заключающееся в изменении системы распределения адресов между банками памяти. Прием чередования адресов базируется на свойстве локальности по обращению, согласно которому последовательный доступ в память обычно производится к ячейкам, имеющим смежные адреса.

Поскольку в каждом такте на шине адреса может присутствовать адрес только одной ячейки, параллельное обращение к нескольким банкам невозможно, однако оно может быть организовано со сдвигом на один такт. Адрес ячейки запоминается в индивидуальном регистре адреса, и дальнейшие операции по доступу к ячейке в каждом банке протекают независимо. При большом количестве банков среднее время доступа к ОП сокращается почти в В раз (В — количество банков), но при условии, что ячейки, к которым производится последовательное обращение, относятся к разным банкам.

7. Оперативная память произвольного доступа(типа RAM – Random Access Memory). Статическая и динамическая памяти (SRAM и DRAM – static/dynamic random access memory).

Основная память - единственный вид памяти к которой ЦП может обращаться непосредственно. Основная память – ЗУ с произвольным доступом.

Основная память может включать в себя два типа устройств:

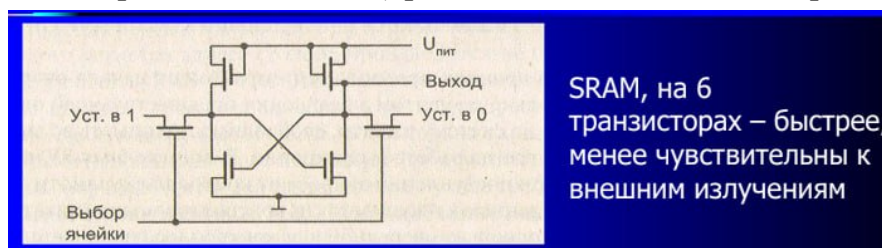
- оперативные запоминающие устройства(ОЗУ)
- постоянные запоминающие устройства(ПЗУ)

ОЗУ – RAM(Random Access Memory). Для большинства типов полупроводников ОЗУ характерна энергозависимость: даже при кратковременном прерывании питания хранимая информация теряется. Микросхема ОЗУ должна быть постоянно подключена к источнику питания и поэтому используется как временная память.

Можно подразделить на две основные подгруппы:

- Динамическую память(DRAM - Dynamic Random Access Memory)
- Статическая память(SRAM - Static Random Access Memory)

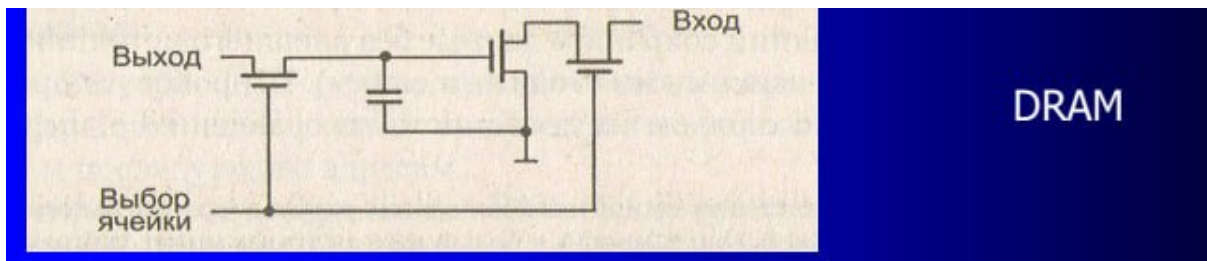
В статических ОЗУ запоминающий элемент хранит записанную информацию неограниченно долго (при наличии питающего напряжения).



SRAM обеспечивает быстрый доступ к данным. Он имеет более высокую скорость чтения и записи по сравнению с DRAM, что делает его предпочтительным для применения в кэш-памяти уровня L1 и L2 на процессорах. Каждая ячейка SRAM обычно требует больше места на кристалле по сравнению с ячейками DRAM, что делает SRAM более дорогостоящим в производстве.

Динамические ОЗУ способны хранить информацию только в течение достаточно короткого промежутка времени, после которого информацию нужно восстанавливать заново, иначе она будет потеряна. Динамические ЗУ, как и статические, энергозависимы. Простота схемы DRAM позволяет достичь высокой плотности размещения, в итоге, снизить стоимость. Главный и большой недостаток – заряд на

конденсаторе со временем теряется. Среднее время утечки заряда DRAM составляет сотни(десятки) миллисекунд, поэтому, заряд следует регенерировать. ВОПРОС ПО РЕГЕНЕРАЦИИ ВОПРОС НОМЕР 9.



8. Управление оперативной памятью (ОП). Организация и функционирование синхронной и асинхронной статической памяти. Организация и функционирование динамической ОП SIMM/DIMM (single/double in-line memory module). (Рогожкина)

Главная задача компьютерной системы – выполнять программы. Программы вместе с данными, к которым они имеют доступ, в процессе выполнения должны (по крайней мере частично) находиться в *оперативной памяти*. Операционной системе приходится решать задачу распределения памяти между пользовательскими процессами и компонентами ОС. Эта деятельность называется управлением памятью. Таким образом, память (storage, memory) является важнейшим ресурсом, требующим тщательного управления.

В микросхемах, где реализован **синхронный** принцип, процессы чтения и записи (если это ОЗУ) выполняются одновременно с тактовыми сигналами контроллера памяти. **Асинхронный** принцип предполагает, что момент начала очередного действия определяется только моментом завершения предшествующей операции. Переноса этот принцип на систему памяти, необходимо принимать во внимание, что контроллер памяти всегда работает синхронно. В асинхронных ЗУ цикл чтения начинается только при поступлении запроса от контроллера памяти, и если память не успевает выдать данные в текущем такте, контроллер может считать их только в следующем такте, поскольку очередной шаг контроллера начинается с приходом очередного тактового импульса.

Тип оперативной памяти важен постольку, поскольку технология изготовления и физические принципы ее функционирования определяют самый важный параметр – быстродействие. Чем выше быстродействие ОП, тем меньше время доступа к ней. В настоящее время наиболее

распространены микросхемы памяти двух типов: статические ОЗУ – SRAM и динамические – DRAM.

Разумеется, более быстрая память дороже стоит, поэтому SRAM используется, как правило, для кэш памяти, в регистрах микропроцессора и системах управления RDRAM.

Динамическое ОЗУ со времени своего появления прошло несколько стадий роста и продолжает совершенствоваться. Вначале микросхемы динамического ОЗУ производились в DIP-корпусах. Затем их сменили модули SIPP, DIMM, SIMM и RIMM.

SIMM - модуль (Single In-Line Memory Module), модуль с однорядным расположением выводов, могут иметь объем 256 Кбайт, 1,2,4, 8, 16 и 32 Мбайт. Модули SIMM для соединения с системной платой имеют не штырьки, а позолоченные полоски (так называемые pin, пины). Первыми SIMM-модулями были 30-пиновые SIMM FPM DRAM, с частотой работы 29 МГц, затем 72-пиновые EDO RAM с частотой 50 МГц. Существенной особенностью ПК, собранных на Pentium является то, что они имеют 64-разрядную шину данных, а это означает, что 32-разрядные SIMM можно устанавливать только парами. В настоящее время SIMM-модули, как 30-пиновые, так и 72-пиновые заменяются модулями DIMM. DIMM (Dual Inline Memory Module) – модуль памяти с двойным расположением 168 выводов. Следует отметить, что разъем DIMM имеют много разновидностей DRAM.

9. Временные диаграммы записи/чтения, регенерации. Принципы работы микросхем DDR(DoubleData Rate), RAMBUS. (Лукашенок А.)

Три типа регенерации:

- одним сигналом RAS (ROR – RAS Only Refresh)

ROR использовалась с первых DRAM. На ШД подаётся адрес регенерируемой строки и RAS – ячейки выбранной строки по внутренним цепям микросхемы записываются обратно. Так как сигнал CAS не появляется, то цикл чтения/записи не начинается.

Недостаток – ША занята в момент регенерации – доступ к другим устройствам ВМ по шине блокирован.

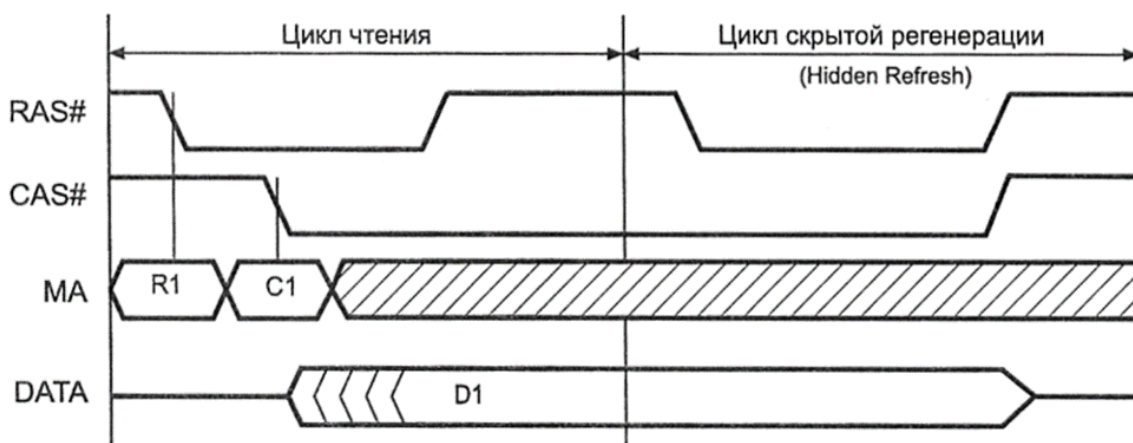
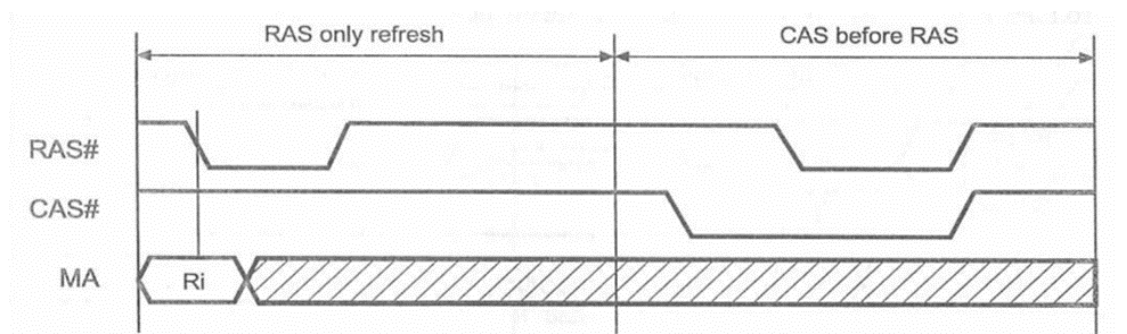
- сигналом CAS, предваряющим RAS (CBR – CAS Before RAS)

CBR – если поступает сначала RAS а потом CAS то это цикл чтения/записи, если наоборот – то цикл регенерации. Адрес строки не передаётся, ИМС использует внутренний счётчик – по кругу.

Плюс – ШД не занята в момент регенерации – доступ к другим устройствам ВМ по шине допускается.

- автоматическая регенерация (SR – Self Refresh)

Автоматическая регенерация – режим энергосбережения (ЦП не работает, тактовый генератор – тоже). При отсутствии внешних сигналов RAS и CAS обновление ROR и CBR невозможно -> ИМС запускает собственный генератор ТИ и производит регенерацию самостоятельно.

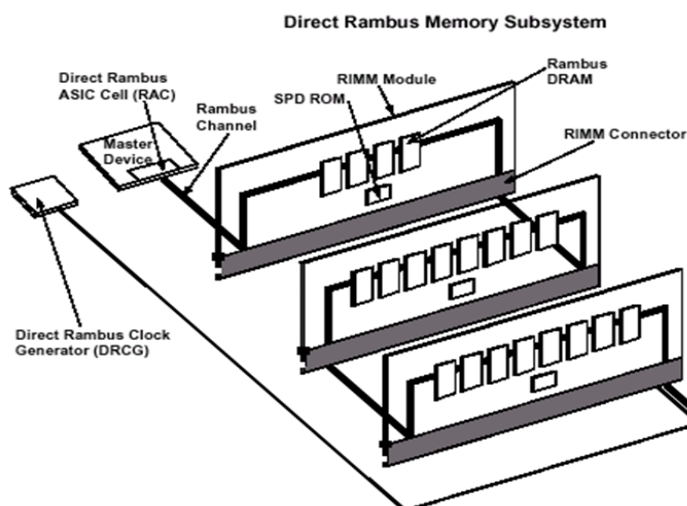


Принцип работы DDR - данные передаются как при восходящем, так и при нисходящем фронтах тактового сигнала, что позволяет удвоить скорость передачи данных по сравнению с традиционными SDR (Single Data Rate) микросхемами. Микросхемы DDR имеют встроенные схемы управления ожиданием и передачей данных, что позволяет им работать на более высоких частотах и с более низкой потребляемой мощностью.

Принцип работы RAMBUS

Технология RAMBUS в 99 году повысила производительность системы за счет использования высокоскоростной специальной шины для передачи данных между RAM и ЦПУ. Эта шина RAMBUS передает 16-разрядные данные с частотой 800 МГц. В технологии RAMBUS используется специальная микросхема памяти – RIMM (Rambus inline Memory mode) линейный модуль памяти RAMBUS. Модуль RIMM вставляется в специальное гнездо, имеющее 168 контактов. Уникальная особенность шины RIMM состоит в том, что с каждым её гнездом должен быть соединён модуль RIMM или специальный модуль связности, который обеспечивает целостность шины.

Вся подсистема состоит из следующих компонентов: основной контроллер (RMC - Rambus Memory Controller), канал (RC - Rambus Channel), разъем для модулей (RRC - Rambus RIMM Connector), модуль памяти (RIMM - Rambus In-line Memory Module), генератор дифференциальных синхроимпульсов (DRCG - Direct Rambus Clock Generator) и сами микросхемы памяти (RDRAM - Rambus DRAM).



10. Контроллер SPD (Serial Presence Detect), опции контроллера и его взаимодействие с операционной системой (ОС). (Лукашенок А.)

Начиная с модулей DIMM, используют последовательный способ идентификации (Serial Presence Detect – SPD), при котором на плату модуля устанавливается специальная дополнительная микросхема, так называемый SPD-чип, представляющая собой небольшую постоянную память на 128 или 256 байт с последовательным (I2C) интерфейсом доступа. В этой микросхеме записана основная информация об изготовителе определенный советом JEDEC (Joint Electron Devices Engineering Council)

Опции контроллера SPD включают такие параметры, как состояние распределения ограничений энергопотребления, определение поддержки DMA, добавление параметров тестирования памяти, оптимизация работы с аппаратными устройствами

Большинство BIOS-ов имеют поддержку SPD, и, когда компьютер загружается, BIOS считывает информацию, занесенную в SPD памяти, и настраивает настройки памяти в соответствии с указанными параметрами.

11. Оперативная память последовательного доступа (ППД). Стековое ЗУ, буфер FIFO (First InFirst Out). (Шиян)

Память (ЗУ) с последовательным доступом [sequential access storage] - Вид памяти (ЗУ), в котором последовательность обращенных к ним входных сообщений и выборок данных соответствует последовательности, в которой организованы их записи.

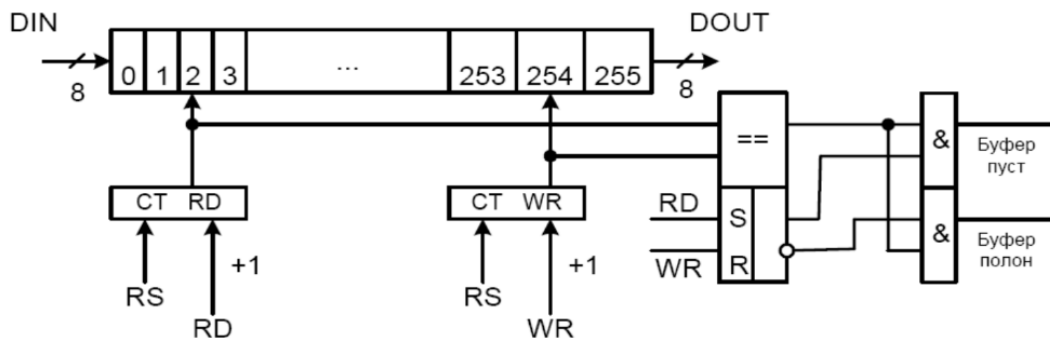
Последовательный доступ представляет собой:

Хранение информации в виде последовательности блоков данных, называемых записями. Для доступа к нужному элементу (слову или байту) необходимо прочитать все предшествующие ему данные. (Магнитные ленты).

Стековые ЗУ - Стековая память состоит из ячеек, связанных друг с другом разрядными цепями передачи слов. Обмен информацией всегда выполняется только через верхнюю ячейку – *вершину стека*. При записи нового слова (команды, числа, символа) все ранее записанные слова сдвигаются на одну ячейку вниз, а новое слово помещается на вершину стека. Считывание возможно только с вершины стека и производится с удалением или без удаления считываемого слова. Такую память часто называют памятью типа LIFO (Last – In First – Out последним вошел, первым вышел).

FIFO – двухпортовое ОЗУ, но один порт для занесения информации, а другой для считывания. Арбитраж – аналогичен. Отличия – нет входов указания адреса

Необходима логика слежения за состоянием очереди – регистры-указатели адресов начала и конца очереди + доп. флаги – отсутствие данных (блок чтения)+полное заполнение (блок записи).



Устройство управления (УУ) — часть центрального процессора, которая вырабатывает распределенную во времени и пространстве последовательность внутренних и внешних управляющих сигналов, обеспечивающих выборку и выполнение команд программы.

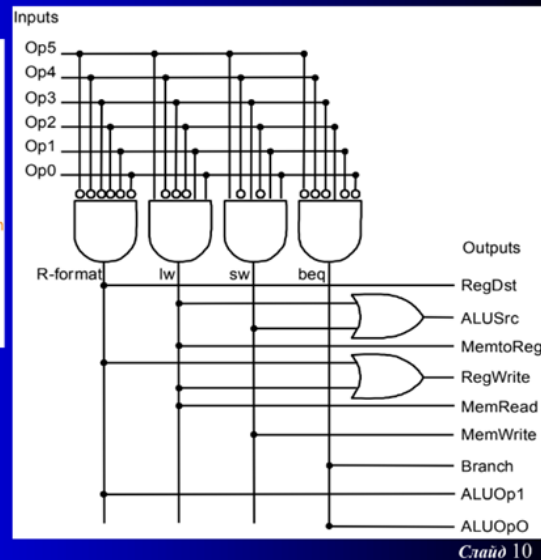
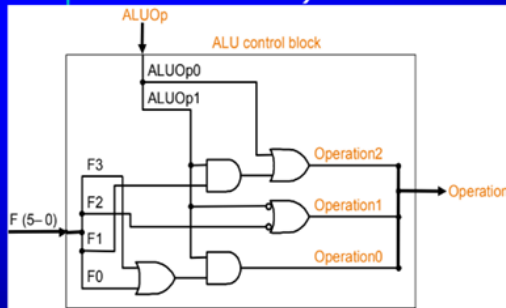
На этапе выборки команды УУ интерпретирует команду, выбранную из оперативной памяти.

На этапе выполнения команды УУ формирует микропрограмму. Микропрограмма - это последовательность микрокоманд, соответствующих одной команде. Эти микрокоманды задают последовательность простейших низкоуровневых операций для АЛУ (пересылка данных, сложение, сдвиг данных, установка и анализ признаков, запоминание результатов и др.)

Устройство управления (УУ) имеет в своем составе три блока: **регистр команды**, который содержит код команды во время ее выполнения **программный счетчик**, в котором содержится адрес следующей, подлежащей выполнению команды, **регистр адреса**, в котором вычисляются адреса операндов, находящихся в оперативной памяти.

Управление

- Простая комбинационная логика (таблица истинности)



Слайд 10

13. Управляющие автоматы (УА) с жесткой и гибкой логикой. Принцип построения УА с программируемой логикой. Принудительная адресация в автомате Уилкса.

Виды структуры УА:

1. **УА с жесткой (схемной, произвольной) логикой**, при которой переключательные функции, необходимые для формирования заданной последовательности управляющих сигналов У, реализуются с помощью логических элементов с произвольными связями (обычно с применением схем с малой и средней степенями интеграции). Здесь используется аппаратный подход к реализации устройства.
2. **УА с хранимой в памяти (гибкой, программной) логикой**, при которой сигналы У вырабатываются на основе совокупности управляющих слов, хранимых в памяти автомата. В этом случае составленные микропрограммы используются в явной форме и обычно записываются в постоянные запоминающие устройства (ПЗУ), выполненные на основе полупроводниковых БИС большой емкости, что позволяет обеспечить регулярность структуры УА и его

компактность; здесь используется аппаратно-программный подход к реализации устройства.

3. **Автомат с программируемой логикой (хранимой в памяти).**

Алгоритм работы записывается в управляющую память в виде последовательности управляющих слов – микрокоманд.

Микрокоманда содержит информацию о микрооперациях, которые должны выполняться в данном такте работы устройства, и об адресе микрокоманды, которая будет выполняться в следующем такте.

Такие автоматы отличаются большой регулярностью структуры и возможностью оперативного внесения изменений в алгоритм работы проектируемого устройства.

Для описания микропрограмм выполнения операций чаще всего используется язык операторных схем алгоритмов. Множество входных сигналов управляющего автомата $X=\{x_1, x_2, x_3\}$ определяется множеством логических условий; множество выходных сигналов управляющего автомата $Y=\{y_1, y_2, \dots, y_k\}$ – множеством управляющих сигналов, используемых для возбуждения микроопераций в операционном автомате, а закон преобразования входных сигналов в выходные определяется микропрограммой.

В микропрограммных УА широко используются два основных способа формирования адреса следующей микрокоманды – естественная (последовательная) и принудительная адресации.

Способ адресации определяет структуру УА, поэтому различают УА с естественной и принудительной адресацией микрокоманд.

При естественной адресации, если МК следуют последовательно в естественном порядке, адрес следующей МК принимается равным адресу предыдущей МК, увеличенному на единицу: $A := A + 1$. В этом случае МК содержит только операционную часть. Естественный порядок следования адресов может быть нарушен, т. е. может возникнуть необходимость перехода к микрокоманде с адресом $A \neq A + 1$. Переход может быть безусловным или зависеть от текущих значений логических условий. Для реализации в микропрограмме таких переходов в микрокоманде должна быть адресная часть. При естественной адресации обычно используются микрокоманды двух типов: операционные и управляющие.

Для **принудительной** адресации адрес следующей микрокоманды указывается в каждой микрокоманде с возможностью его модификации в

зависимости от значений логических условий. Адрес следующей микрокоманды задается безусловно или выбирается по условию, определяемому текущими значениями логических условий. Возможная структура микрокоманды :

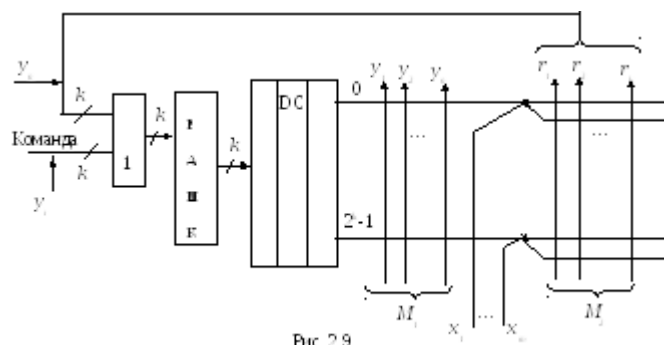
АПХ1Х2У1У2056891112171823

Формат микрокоманды включает пять полей:

- АП(0:5) – адрес перехода, причем 4 и 5 разряды используются для принудительной модификации адреса;
- Х1(6:8) и Х2(9:11) – поля логических условий.;
- У1(12:17) и У2(18:23) – поля управляющих сигналов.

Автомат Уилкса относят к классу автоматов с хранимой логикой.

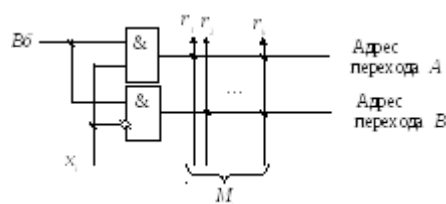
Структурная схема такого устройства имеет следующий вид:



Здесь $M1$ – матрица (ПЗУ) управляющих сигналов; $M2$ – матрица (ПЗУ) адресов переходов.

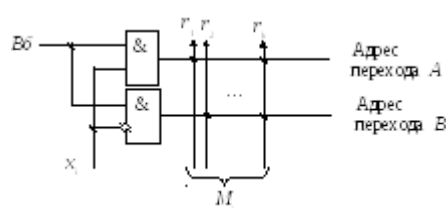
Линия выборки дешифратора адресует строку матрицы $M1$ и пару строк матрицы $M2$. Таким образом, схема $M2$ содержит в 2 раза больше строк, чем ПЗУ матрицы $M1$.

Блок выборки следующего адреса в схеме автомата Уилкса может быть представлен в следующем виде:



Приведенная реализация поясняет принцип ветвления в микропрограмме. В частности, элементы 2И осуществляют анализ признака x_i и выбор соответствующего адреса из ячейки A или B ПЗУ. При этом, если $x_i = 0$, адрес извлекается из ячейки A , при $x_i = 1$ – из ячейки B . Для организации условных переходов содержимое ячейки A и ячейки B должно быть различным. Если же в микропрограмме реализуется безусловный переход, то содержимое строки A и строки B ПЗУ **должно быть одинаковым.

Функциональная схема автомата Уилкса может быть представлена в виде



В данной схеме ПЗУ1 формирует сигналы управления, ПЗУ2 – адрес следующей микрокоманды, номера условий перехода определяет состояние входа I MUX. При запуске автомата команда с помощью сигнала u_i заносится в РАМК и адресует первую микрокоманду выполняемой микропрограммы. УУ формирует сигналы управления вычислительным процессом $y^1, y^2, \dots, y^n$ один из которых (y^n) – используется для разрешения записи в регистр адреса микрокоманды (РАМК) следующего $k+1$ разрядного адреса с выхода ПЗУ2. В процессе функционирования младший разряд адреса $k+1$ ПЗУ2 используется для выбора одной из двух ячеек памяти, адресуемых k разрядами РАМК. Иными словами этот разряд конкретизирует адрес перехода при выборке следующей микрокоманды в процессе функционирования устройства.

14. Блок микропрограммного управления. Типы микрокоманд. Кодирование полей микрокоманды. Микропроцессорная секция. (Семков)

15. Микропрограммирование. Горизонтальный, вертикальный и смешанный способы кодирования микроопераций. (Кулакович)

Компьютеры состоят из нескольких уровней. Один из таких уровней называется *микроархитектурным уровнем*. На этом уровне можно увидеть

совокупности из 8 или 32 регистров, которые формируют локальную память и схему, называемую АЛУ (Арифметико-логическое устройство). Регистры вместе с АЛУ формируют тракт данных, по которому поступают данные. Основная операция тракта данных состоит в следующем: выбирается два или один регистр, АЛУ производит над ним какую-либо операцию, а результат помещается в один из выбранных регистров. Следует помнить, что на некоторых машинах этот уровень реализован аппаратно, а не программно.

На некоторых машинах работа тракта данных контролируется **микропрограммой**. Микропрограмма вызывает команды из памяти и выполняет их одну за другой, используя тракт данных.

Информация о том, какие сигналы управления должны быть сформированы в процессе выполнения текущей микрокоманды, в закодированном виде содержится в микрооперационной части микрокоманды. Способ кодирования микроопераций определяет сложность аппаратных средств устройства управления и его скоростные характеристики. Применяемые в микрокомандах варианты кодирования сигналов управления можно разделить на три группы:

- Минимальное кодирование (горизонтальное микропрограммирование)
- Максимальное кодирование (вертикальное микропрограммирование)
- Групповое кодирование (смешанное микропрограммирование)

При горизонтальном микропрограммировании под каждый сигнал управления в микрооперационной части команды выделен один разряд, который позволяет в рамках одной микрокоманды формировать различные сочетания сигналов управления, что обеспечивает максимальный параллелизм выполнения микроопераций. Самый главный минус этого типа микропрограммирования заключается в высоких затратах на хранение микрооперационных частей микрокоманд.

При вертикальном микропрограммировании каждой микрооперации присваивается некоторый код, который заносится в микрооперационную часть. Соответственно, микрооперационная часть получается минимальной. Такой способ кодирования требует минимальных аппаратных затрат на хранение микрокоманд. Однако придется использовать дешифратор, чтобы преобразовывать код микрооперации в соответствующие сигналы управления. При большом количестве сигналов управления дешифратор вносит большую задержку, инициирующий только одну микрооперацию, за счет чего увеличивается длина микрокоманды и время на ее исполнение.

При смешанном микропрограммировании микрооперации разбивают на группы, вследствие чего появляются еще два типа: горизонтально-вертикальное и вертикально-горизонтальное программирование.

В горизонтально-вертикальном методе в каждую группу включаются взаимно несовместимые сигналы управления (микрооперации), то есть сигналы управления, которые никогда не встречаются вместе в одной микрокоманде. При этом сигналы, обычно формируемые в одном и том же такте, оказываются в разных группах. Внутри каждой группы сигналы управления кодируются максимальным (вертикальным) способом, а группы — минимальным (горизонтальным) способом.

При вертикально-горизонтальном способе все множество сигналов управления (микроопераций) также делится на несколько групп, однако в группу включаются сигналы управления (микрооперации), наиболее часто встречающиеся вместе в одном такте. Поле микрооперации делится на две части: МО и МОгр. Поле МО, длина которого равна максимальному количеству сигналов управления (микроопераций) в группе, кодируется горизонтально, а поле МОгр, указывающее на принадлежность к определенной группе, — вертикально.

(Да душно, какой вопрос такой и ответ)

16. Естественная адресация микрокоманд. Функциональная схема управляющего автомата с естественной адресацией.

В автоматах с программируемой логикой используются, в основном, два метода адресации микрокоманд: принудительная и естественная.

При принудительной адресации адрес перехода к следующему двоичному слову в явном (или неявном) виде присутствует в самой микрокоманде. Так, в рассмотренных выше примерах, адресация была принудительная. В двоичном слове для хранения адресов переходов отводилось два поля (при одном логическом условии в слове). Такая микрокоманда называется двуадресной.

Естественная адресация предполагает наличие двоичных слов разного вида, например, операционные и управляющие слова. Операционные слова соответствуют операторным вершинам ГСА управляющего автомата, а управляющие — условным. Если в ГСА автомата есть линейные части (не содержащие условных вершин), то адреса слов, описывающих следующие друг за другом операторные вершины ГСА, должны формироваться в естественном нарастающем порядке: $a_{t+1} = a_t + 1$,

где: a_t — адрес текущего слова,

a_{t+1} — адрес следующего слова.

Управляющее слово может быть эквивалентно управляющей конструкции некоторого языка программирования:

IF $X_i = 1$ THEN GOTO $a_{xi=1}$;

если значение логического условия $X_i = 1$, то адрес следующего слова $a_{t+1} = a_{xi=1}$; если $X_i = 0$, то адрес следующего слова: $a_{t+1} = a_t + 1$.

Формат микрокоманд (двоичных слов) при естественной адресации может быть следующим. Операционная микрокоманда содержит два поля: поле признака вида микрокоманды P (операционная или управляющая; пусть для операционной – $P=0$) и поле, в котором любым из возможных способов записаны значения u_i , которые должен вырабатывать автомат. Управляющая микрокоманда содержит три поля: поле признака P (пусть для управляющей – $P=1$), поле логического условия (может содержать № логического условия X_i) и поле адреса перехода при $X_i = 1$.

17. Типовая структура блока обработки данных ЭВМ.

Блок обработки данных - это CPU, главный рабочий элемент ЭВМ, который выполняет команды программ и управляет действиями других ее узлов. Основными составными частями ЦП являются: арифметико-логическое устройство (АЛУ), устройство управления (блок команд), память (ПЗУ) а также регистры - функционально ориентированные ячейки памяти - в том числе регистр команд, накапливающий регистр и регистр последовательного управления.



18. Арифметико-логические устройства (АЛУ), матричные умножители, множительно-суммирующие блоки, регистры и регистровые файлы.

АЛУ предназначено для арифметической и логической обработки данных. Обычно состоит из регистров, сумматоров с соответствующей логикой и элемента управления выполняемым процессом. **Множительносуммирующие блоки** в отличие от множительных блоков добавляют к результату два слагаемых, одно из которых разрядности множимого, а другие - множителя. Итоговое значение формируется по формуле:

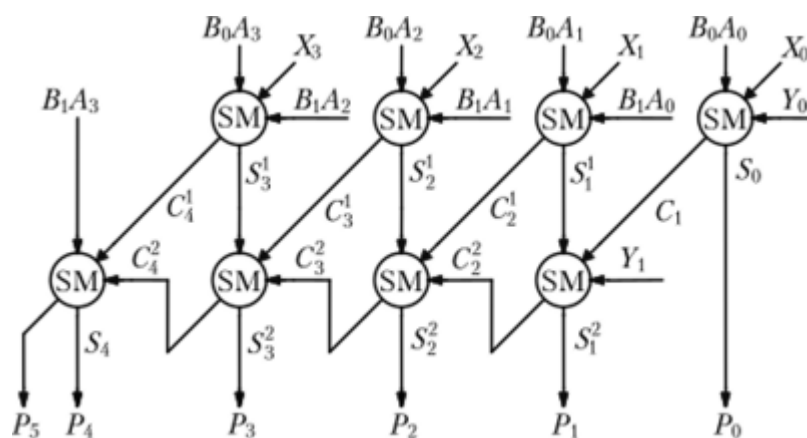
$$P = A(m) * B(n) + X(m) + Y(n)$$

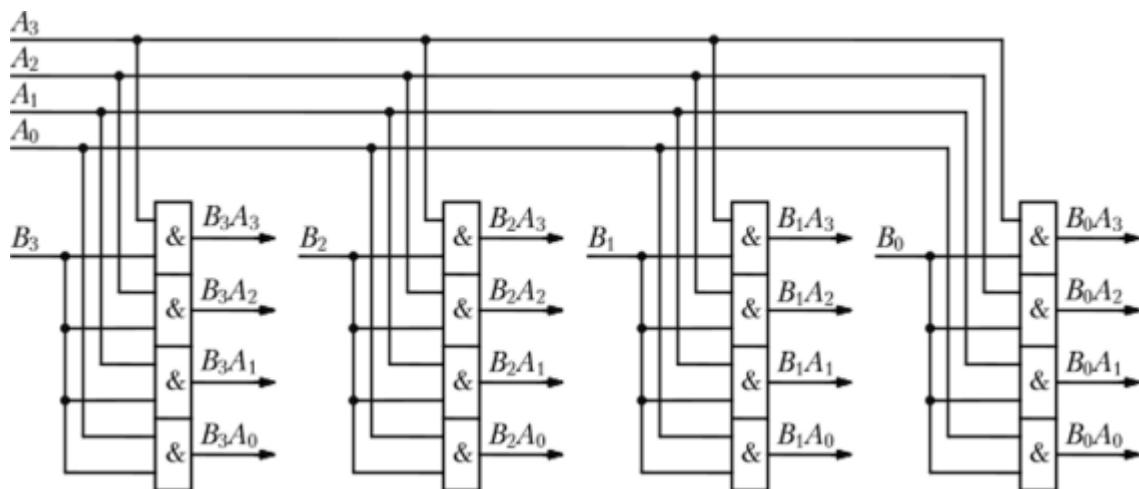
принцип организации в мн-сумм блоке 4х2 с использованием одноразрядных сумматоров

2^5	2^4	2^3	2^2	2^1	2^0
		X_3	X_2	X_1	X_0
	+	B_0A_3	B_0A_2	B_0A_1	B_0A_0
+	+	B_1A_3	B_1A_2	B_1A_1	B_1A_0
				Y_1	Y_0
P_5	P_4	P_3	P_2	P_1	P_0

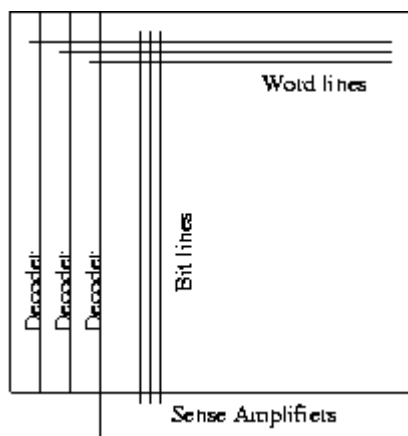
особенности:

- появление членов X_3, X_2, X_1 привело к использованию полных одноразрядных сумматоров (вместо неполных сумматоров, используемых в множительном блоке);
- члены X_0, Y_0, Y_1 потребовали дополнительно двух полных одноразрядных сумматоров





Регистровый файл (register file) — модуль микропроцессора (CPU), содержащий в себе реализацию регистров процессора. Современные регистровые файлы, используемые в СБИС, обычно реализованы как многопортовый массив быстрой статической памяти [SRAM](#). Традиционно регистровый файл реализуется как массив запоминающих ячеек, считываемый вертикально. Слова расположены в горизонтальных строках (word lines), и при чтении ячейки выдают своё значение на вертикальные битовые линии (Bit lines). В нижней части эти линии подключены к усилителям, которые преобразуют сигналы с ячеек, ослабленные при передаче, в булевы сигналы полной амплитуды. В левой части расположены декодеры, активирующие строку, соответствующую заказанному регистру или регистрам. Большие регистровые файлы иногда организуются как мозаика из нескольких отраженных и повернутых более простых регистровых файлов.



19. Разработка операционных устройств (ОУ). Проектирование операционных автоматов (ОА) с фиксированной структурой. (Ледохович)

20. Синтез ОА с управляющей логикой. Структурная организация и синтез ИМ-автоматов. Использование ЗУ в автоматах.

1) *Исходной информацией для разработки структурной схемы ОА являются:*

- описание типов слов, используемых в микропрограммах;
- список микроопераций;
- список логических условий.

Все слова (переменные), кроме вспомогательных, должны сохранять свое значение неограниченно долго, вплоть до присвоения им нового значения. Такие переменные являются **переменными с памятью** в отличие от вспомогательных переменных - переменных без памяти. Переменные с памятью соответствуют всем внутренним (L) словам.

Каждой переменной с памятью ставится в соответствие регистр операционного автомата той же разрядности, что и переменная, причем i -й разряд регистра соответствует i -му разряду переменной. Регистру присваивается идентификатор соответствующего ему слова.

Каждой переменной без памяти ставится в соответствие шина, имеющая разрядность переменной и предназначенная для параллельной передачи слов.

Для каждого регистра определяется множество микроопераций Y_s и логических условий X_s . На основании анализа микроопераций Y_s и логических условий X_s производится выделение всех полей регистра S . В одно поле включаются те разряды регистра S , которые преобразуются микрооперациями Y_s одинаково.

2) *Структура ИМ-автомата (укрупненно) порождается следующим образом.*

1) Аналогично, как и для М-автомата, определяются все операторы, реализуемые в комбинационных схемах АЛУ.

2) Определяются классы операторов, эквивалентных в смысле выполняемых ими функций.

3) В состав ОА включаются комбинационные схемы АЛУ, реализующие определенные совокупности эквивалентных операторов в форме обобщенного оператора.

Таким образом, ИМ-автоматы можно рассматривать как композицию из k М-автоматов, взаимодействующих с общим полем регистров. Исходя из этого, синтез автоматов среднего класса сводится к разбиению всего множества

микрокоманд на k подмножеств и синтезу k М-автоматов, реализующих функции данных подмножеств.

Этапы синтеза ИМ-автомата:

- 1 Составление списка микрокоманд, выполняемых в ОА.
- 2 В зависимости от типа выполняемой микрокоманды и модифицируемого числа операндов множество микрокоманд Y разбивается на два (по числу АЛУ) подмножества. В первое Y_1 включаются, например, бинарные преобразования, Во второе подмножество Y_2 включаются унарные преобразования.
- 3 Определяется разрядность слов, передаваемых на входы A_1 и A_2 АЛУ₁ и вход A_3 АЛУ₂.
- 4 Составляется список операторов, реализуемых схемами АЛУ₁ и АЛУ₂.
- 5 Составляется таблица кодирования микрокоманд сигналами управления.
- 6 Определяются классы эквивалентных операторов.
- 7 Для подмножеств эквивалентных операторов строятся обобщенные операторы.
- 8 Формирование списка логических условий.
- 9 Строится структурная схема ИМ-автомата путем суперпозиции подсхем.
- 10 Верификация суперскалярного устройства с возможным возвратом на предыдущие этапы проектирования для внесения коррективов, обусловленных общим ходом синтеза схемы.

3) *Запоминающие устройства*, используемые в автоматах с программируемой логикой, имеют достаточно большую емкость, а значит и большую длину адресной части микрокоманды. Большая емкость ЗУ позволяет хранить в нем не одну, а множество микропрограмм, реализующих разные функции, причем каждой из них требуется небольшой объем памяти.

Для того, что бы максимально использовать ЗУ и при этом не увеличивать длину адресной части микрокоманды используют прием, который называется «сегментация ЗУ». Весь объем ЗУ делится на сегменты (страницы), а адрес A , поступающий на адресный вход ЗУ рассматривается как адрес сегмента A_c (обычно старшие разряды адреса) и адрес ячейки в сегменте A_y (обычно младшие разряды адреса). Число сегментов обычно выбирают равным целой степени двойки (2, 4, 8 и т.д. сегментов).

21. Способы взаимодействия процессора с внешними устройствами.(Семков)

22. Команды ввода-вывода. Буферизация вывода и отложенный обмен (опции WT/WB в BIOS - Basic Input-Output System). Порты ввода-вывода и адресные пространства ввода-вывода.

Буферизация позволяет процессам, производящим ввод, вывод и обработку данных, выполняться параллельно, не ожидая, пока другой процесс выполнит свою часть работы. Отложенный обмен (асинхронный обмен) – метод обмена данными, при котором отправитель и получатель не обязаны работать синхронно.

WT (Write-Through) и WB (Write-Back) — это две основные политики записи, используемые в кэше.

Write-Through (WT):

- Данные одновременно обновляются в кэше и в памяти.
- Этот процесс проще и надежнее.
- Используется, когда записей в кэш производится нечасто (количество операций записи меньше).
- Помогает в восстановлении данных (в случае отключения питания или сбоя системы).

Write-Back (WB):

- Данные сначала записываются в кэш, а затем, при определенных условиях (например, при замещении данных), обновляются в памяти.
- Может уменьшить количество обращений к памяти, что положительно сказывается на производительности.
- Возможны проблемы с согласованностью данных.
- Не всегда может помочь восстановить данные.

x86 – отдельная адресация памяти и портов(модулей) ввода-вывода.

Сигналы чтения и записи из/в порт на шине ISA – IORD# и IOWR# (чтение/запись из/в память MEMRD# MEMWR#) – порождают шинные циклы обмена чтения из порта и записи в порт.

На шине PCI – разделение иначе – тип операции кодируется 4 байтной командой, в зависимости от типа инструкции, выполняемой ЦП.

Пространство ввода/вывода (BB) – 64kB, -> адресация по 16 линиям. Изначально использовались лишь 10 бит адреса шины ISA (адреса 0-3FFh). Сейчас в адаптерах для шин MCA и PCI – используются все 16 бит адреса.

23. Системная магистраль ISA (Industry Standard Architecture), назначение сигналов, протоколы обмена на магистрали. Блок прямого доступа к памяти в системной логике чипсета. (Тюшкевич)

Системная магистраль ISA (Industry Standard Architecture) - это 8- или 16-разрядная шина ввода-вывода IBM PC-совместимых компьютеров. Служит для подключения плат расширения стандарта ISA. Конструктивно выполняется в виде 62- или 98-контактного разъёма на материнской плате.

ISA— Industry Standard Architecture, известная также как 8-разрядная PC/XT или 16-разрядная AT-Bus.

Пересылка: одиночная или блочная

Арбитраж: централизованный

Расщепление транзакций: нет

Протокол: синхронный

Тактовая частота: 4,77/8,33 МГц

Полоса пропускания, один. пересылка: 33 Mbyte/c

Назначение сигналов

Шина ISA имеет следующие сигналы:

1. Шина адреса (Address bus) - 20 или 24 бита, используется для адресации памяти или устройств.
2. Шина данных (Data bus) - 8 или 16 бит, используется для передачи данных.

Протоколы обмена на магистрали

Обмен данными на шине ISA осуществляется по следующим протоколам:

1. DMA (Direct Memory Access) - прямой доступ к памяти.
2. Interrupt (Прерывание) - передача данных по прерыванию.
3. Polling (Опрос) - передача данных по опросу.

Блок прямого доступа к памяти в системной логике чипсета

Блок прямого доступа к памяти (DMA) в системной логике чипсета отвечает за управление обменом данными между процессором и памятью или устройствами.

Блок DMA имеет следующие функции:

1. Адресация памяти
2. Счетчик адреса
3. Управление передачей данных

Блок DMA работает следующим образом:

1. Процессор инициирует обмен данными с памятью или устройством.
2. Блок DMA получает от процессора адрес памяти или устройства.
3. Блок DMA начинает передачу данных.
4. Процессор может продолжать выполнение других операций.

Блок DMA позволяет процессору выполнять другие операции, пока происходит обмен данными с памятью или устройством. Это повышает эффективность работы процессора.

Достоинства и недостатки шины ISA

Достоинства шины ISA:

1. Простота реализации
2. Низкая стоимость
3. Широкая поддержка периферийных устройств

Недостатки шины ISA:

1. Низкая скорость передачи данных
2. Ограниченное количество устройств
3. Отсутствие поддержки современных периферийных устройств

Шина ISA была разработана в 1981 году и использовалась в IBM PC-совместимых компьютерах в течение многих лет. В настоящее время шина ISA устарела и заменена более скоростными и современными шинами, такими как PCI, PCIe и USB.

24. Контроллер ПДП. Структура, функционирование и программирование контроллера КПП

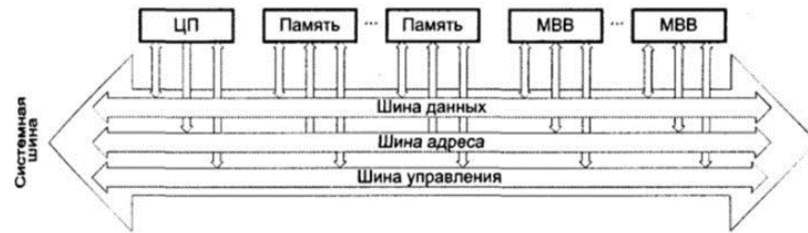
25. Понятие шины ЭВМ. Типы шин. Иерархия шин. Вопросы физической реализации шин. Арбитраж шин. Протоколы шин.(Чепик)

Шина – Совокупность трактов(линий), объединяющих между собой основные устройства ВМ

Типы шин:

- **Шина “процессор-память”** (минимальная длина, максимальная пропускная способность)
 - связь между ЦП и основной памятью Front-Side Bus (FSB)
 - связь процессора с кэш-памятью Back Side Bus (BSB)

- **Шины ввода-вывода** (маленькая пропускная способность, большая длина) Не используется для видеосистем тк они медленные
- **Системная шина** (компромисс между длиной и скоростью)(несколько сот линий)



Иерархия: (Адаптеры – буферизация данных.)

- **ВМ с одной шиной**

Плюс: стоимость, простота

Минус: скорость транзакций



- **ВМ с двумя видами шин**

Плюс: снижает нагрузку на шину “ЦП-память”

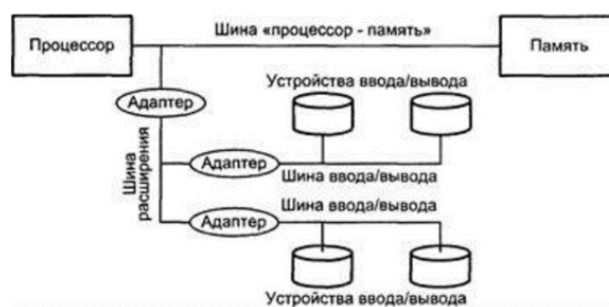
Минус: стоимость



- **ВМ с тремя видами шин**

Плюс: снижает нагрузку на шину “ЦП-память”, улучшает характеристики шины “ЦП-память”

Минус: стоимость



Физическая реализация шин

- **Механические аспекты:**

- Сигнальные линии - тонкие медные полоски на плате.
- Широкие медные полоски - проводящие напряжение.
- Разъёмы - проблема надёжного контакта.
- Механические спецификации - размеры плат, направляющих, максимальная высота элементов и т.п.

- **Электрические аспекты:**

- Драйвер или «возбудитель» шины - схема меняющая напряжение на плате.

- Способ отключения драйвера - через специальный вход.
- Линия шины - к выходу драйвера через резистор от источника питания.
- Способ позволяет реализовывать «монтажное ИЛИ/И» - сигнал на линии - результат сложения (ИЛИ) всех поступивших на линию сигналов.

· **Распространение сигнала:**

- Скорость распространения - теоретически 300 м/с, в реале - не более 70%.
- Отражение - эффекты возникающие при перемене сигнала в линии.
- Перекос сигнала - при параллельной передаче данные поступают не одновременно ко всем соответствующим приёмникам.
- Эффекты перекрестного влияния – перекрёстная или переходная помеха

Арбитраж шин

К шине может быть подключено несколько потенциальных ведущих, но в любой момент времени активным может быть только один из них: если несколько устройств передают информацию одновременно, их сигналы перекрываются и искажаются. Для предотвращения одновременной активности в любой шине предусматривается процедура допуска к управлению шиной только одного из претендентов (арбитраж). Арбитраж осуществляется за счет присвоения приоритетов.

Существует два основных типа арбитража: централизованный и децентрализованный.

При **централизованном арбитраже** в системе имеется специальное устройство, называемое центральным арбитром, которое принимает решение о том, какое из устройств будет иметь доступ к шине. Центральный арбитр может быть самостоятельным модулем или частью центрального процессора.

Плюсы:

- Простота реализации
- Высокая производительность

Минусы:

- Наличие единственной точки отказа
- Децентрализованный арбитраж

При **децентрализованном арбитраже** каждое из устройств, претендующих на доступ к шине, имеет свой собственный блок арбитража. Блоки арбитража взаимодействуют друг с другом, чтобы решить, какое из устройств будет иметь доступ к шине.

Плюсы:

- Отсутствие единственной точки отказа
- Повышенная надежность

Минусы:

- Более сложная реализация
- Более низкая производительность

Статический приоритет – потенциальному ведущему присваивается определенный уровень приоритета, который может оставаться неизменным

Динамический приоритет – приоритет, изменяется по какому-либо алгоритму:

- Простая циклическая смена приоритетов – после каждого цикла – изменение приоритета на единицу по кругу.
- Циклическая смена приоритетов с учётом последнего запроса – последний обслуженный получает самый низкий приоритет, остальные за ним сдвигаются по кругу (более распространена).

- Смена приоритетов по случайному закону – генератор случайных чисел назначает новые значения приоритетов
 - Схема равных приоритетов – при поступлении нескольких запросов – каждый из них имеет шансы на обслуживание, конфликт решается арбитром (обычно асинхронные схемы).
 - Алгоритм наиболее давнего использования – LRU (Last Recently Used) – после каждого цикла – наивысший приоритет – кто дольше всех не использовал шину.
- Доп алгоритмы (не чисто динамические – не после каждого цикла):
- Очередь - FIFO (сложная аппаратура – редко используется)
 - Фиксированный квант времени – каждому ведущему фиксированный промежуток для захвата. (хорошо подходит для шин с синхронным протоколом)

Протоколы шин — набор правил, определяющих, как устройства на шине обмениваются информацией. (метод информирования о достоверности адреса, данных, управляющей информации и информации о состоянии)

Характеристика	Синхронные шины	Асинхронные шины
Привязка к тактовому сигналу	Привязаны к импульсам ГТИ (тактовым импульсам(ТИ)).	Не зависят от тактового сигнала, зависят от синхронизирующих сигналов (стробов).
Скорость передачи данных	Ограничена тактовой частотой. Ведущий должен работать со скоростью самого медленного из устройств, участвующих в пересылке данных	Определяется ведомым устройством, шины самосинхронизирующиеся.
Управление временем	Все устройства должны оперировать в пределах тактового периода.	Управление временем осуществляется с помощью синхронизирующих сигналов, что обеспечивает большую гибкость. Синхросигналы от ведомого – квитирующие сигналы (handshakes) или подтверждения сообщений (acknowledges).
Перекося сигналов	Возможен, но встроен в тактовую частоту.	Проблема перекося требует учета в каждой транзакции и устройстве.
Распространение сигналов	Доставка ТИ к каждому разъему практически одновременно.	Скорость асинхронной пересылки определяется ведомым устройством, что позволяет применять новые и старые устройства.
Квитирование (Handshaking)	Не всегда прямолинейно, ведущий не знает, что ответил ведомый.	Квитирование устанавливается с помощью двунаправленного обмена сигналами управления, что обеспечивает более надежное уведомление о завершении транзакции.

Применение	Обычно используются в "процессор-память" шинах.	Чаще встречаются в шинах ввода/вывода.
------------	---	--

26. Системные шины ISA, FSB – front side bus, шина в/в PCI - Peripheral component interconnect, PCI-Express, назначение сигналов, протоколы обмена. Методы повышения эффективности шин. (Гембар) Сравнение стандартов шин.

Шина ISA конструктивно выполнена в виде двух разъемов. 8-разрядная шина ISA использует 62-контактный слот (ряды A,B), в который вставляются платы расширения, рассчитанные на обмен 8-разрядными данными (8-разрядные платы). 16-разрядная шина ISA занимает два разъема: основной 62-контактный и дополнительный 36-контактный слот (ряды C,D). В них вставляются платы расширения, рассчитанные на обмен как 16- так и 8-разрядными данными (16-разрядные платы).

Назначение сигналов шины

I/ORD# - 8/16, I, 3-стаб - Чтение данных из внешнего устройства

I/OWR# - 8/16, I, 3-стаб - Запись данных во внешнее устройство

AEN - 8/16, I, ТТЛ - Разрешение адресации портов

I/O CHRDY# - 8/16, O, ОК - Готовность канала ввода-вывода

OWS# - 8/16, O, ОК - Отсутствие состояний ожидания

I/O CH16# - 16, O, ОК - Выбор цикла для внешнего устройства

IRQ15, 14, 12, 11, 10 -16; - Сигналы запросов прерываний

RESET DRV - 8/16, I, ТТЛ - Системный сброс

В режиме программного обмена информацией на магистрали ISA выполняется четыре типа циклов:

цикл записи в память;

цикл чтения из памяти;

цикл записи в устройство ввода/вывода;

цикл чтения из устройства ввода/вывода.

FSB (Front Side Bus, системная шина) – это шина (набор сигнальных линий), которая используется для обмена информацией между процессором и устройствами системного блока (оперативная память, порты ввода-вывода, видеокарта, жесткий диск и др.) компьютера.

PCI (Peripheral Component Interconnect) – это компьютерная шина ввода/вывода, предназначена для подключения периферийных устройств к системной плате персонального компьютера. Шина PCI поддерживает 32-х/64-х битный обмен данными.

Сигналы:

AD[31:0] Address/Data — мультиплексированная шина адреса/данных.

FRAME# Кадр. Введением сигнала отмечается начало транзакции

IRDY# Initiator Ready — готовность ведущего устройства к обмену данными

STOP# Запрос ЦУ к ведущему устройству на остановку текущей транзакции

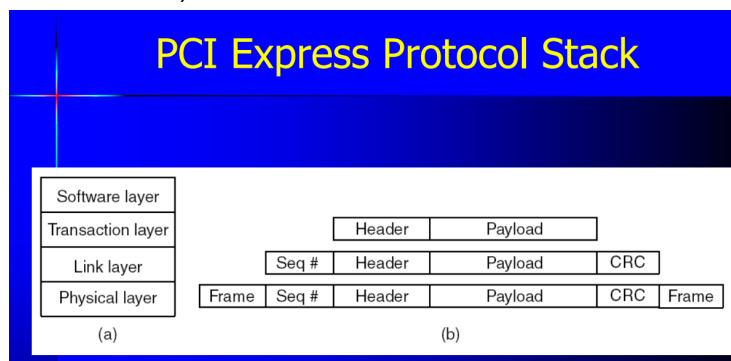
LOCK# Сигнал блокировки (захвата) шины

PERR# Parity Error — сигнал об ошибке четности

RST# Reset — сброс всех регистров в начальное состояние

Новая системная шина общего назначения, называемая **PCI Express**, была предложена летом 2002 года. В отличие от PCI и AGP, новая шина последовательная, а не параллельная. Это даёт снижение стоимости и уменьшение размеров за счёт уменьшения количества соединительных линий и контактов в разъемах.

PCI Express пересылает всю управляющую информацию, включая прерывания, через те же линии, что используются для передачи данных. Последовательный протокол никогда не может быть заблокирован, таким образом задержки шины PCI Express вполне сравнимы с таковыми для шины PCI (заметим, что шина PCI для передачи сигнала о запросе на прерывание использует отдельные физические линии IRQ#A, IRQ#B, IRQ#C, IRQ#D).



Приемы повышения производительности шин:

- Пакетный режим
- Конвейеризация транзакций
- Расщепление транзакций

Пакетный или блочный режим (burst mode) – как для асинхр., так и для синхр-х. Один адресный цикл сопровождается множественными циклами данных (либо чтения либо записи, но не смешанных!). Пакет данных передаётся без указания адресов данных внутри пакета -> увеличение адреса производится автоматически как у ведущего, так и у ведомого. Длина пакета может достигать до 1024 байт.

Конвейеризация транзакций: очередной элемент данных может быть послан устройством А, до того, как устройство В завершит считывание предыдущего элемента.

В протоколе с расщеплением – две шины ША и ШД обязательно разделены. Ведущий выставляет адрес и требование данных. После чего шина свободна. Когда память подготовит данные она становится ведущим и запрашивает доступ к шине, после чего пересылает данные запрашивающему. Соответственно, пока память готовит данные к отправке шина свободна и может использоваться другими ведущими...

Стандарты шин

Шины ввода/вывода

	PCI	SCSI	SCSI-II	IDE
Протокол	Синхр.	Синхр. и асинхр.		Асинхр.
Тактовая частота	33/66 МГц	5/10 МГц	10/20/40/80	
Полоса проп-ния, один. пересылка	33 Mbyte/c	1,5 (асинх) 5 (синх) Mbyte/c	5-40 (синх) Mbyte/c	
Полоса проп-ния, блочн. пересылка	960 Mbyte/c	1200 Mbyte/c	40/80/160/320 (синх) Mbyte/c	до 200
Макс. длина, м	0,5	25	25	0,5
Стандарт	Нет	ANSI X3.131-1986	ANSI X3.131-199x	ANSI X3.T9.2./90-14

Слайд 23

Стандарты шин

Системные шины персональных ВМ

	NuBus	ISA 8/16	EISA	FSB Pentium 4
Вид пересылки	Одиночная или блочная			
Арбитраж	Централизованный			
Расщепление транзакций	Нет	Нет	Возможно	Есть
Протокол	Синхронный			
Тактовая частота	10 МГц	4,77/8,33 МГц	8,33 МГц	400 (баз. 100) 533 (баз. 133) 800 (ожидается)
Полоса проп-ния, один. пересылка	40 Mbyte/c	33 Mbyte/c	33 Mbyte/c	1060 (133) 3200 (400) 4200 (533)

Слайд 21

Стандарты шин

Системные шины высокопроизводительных серверов

	Summit	Challenge	XDBus
Протокол	Синхронный	Синхронный	Синхронный
Тактовая частота	60 МГц	48 МГц	66 МГц
Полоса проп-ния, один. пересылка	60 Mbyte/c	48 Mbyte/c	66 Mbyte/c
Полоса проп-ния, блочн. пересылка	960 Mbyte/c	1200 Mbyte/c	1056 Mbyte/c
Макс. длина, м	0,3	0,3	0,4
Стандарт	Нет	Нет	Нет

Слайд 19

27. Система команд процессора. Классификация систем команд по месту хранения операндов. Классификация систем команд по составу и сложности команд. Адресные структуры основной памяти. (Филичева)

28. Команды вычислительной машины и их размещение в памяти. Слово состояния процессора. Способы адресации операндов в памяти.(Мешков)

набор машинных команд. Они управляют действиями машины. В этом наборе всегда в той или иной форме присутствуют команды LOAD и STORE, предназначенные для перемещения данных между памятью и регистрами, и команда MOVE, которая служит для копирования данных из одного регистра в другой. Также всегда присутствуют арифметические и логические команды, команды сравнения элементов данных и команды переходов в зависимости от результатов.

регистр PSW (Program State Word — слово состояния программы), который еще называют флаговым. Флаговый регистр содержит различные биты, необходимые центральному процессору. Самые важные биты — это коды условий. Они устанавливаются в каждом цикле АЛУ и отражают состояние результата предыдущей операции:

N — результат отрицателен (Negative);

Z — результат равен 0 (Zero);

V — результат вызвал переполнение (oVerflow);

C — перенос самого левого бита (Carry out);

A — перенос бита 3 (Auxiliary carry — служебный перенос);

P — результат четный (Parity).

Флаговый регистр может хранить не только коды условий. Его содержимое в разных машинах может быть разным. Дополнительные поля указывают режим машины (например, пользовательский или привилегированный), бит трассировки (который используется для отладки), уровень приоритета процессора, статус разрешения прерываний. Флаговый регистр обычно читается в пользовательском режиме, но некоторые поля могут записываться только в привилегированном режиме (например, бит, который указывает режим).

Непосредственная адресация

Прямая адресация

Регистровая адресация

Косвенная регистровая адресация

Индексная адресация

Относительная индексная адресация

29. Способы адресации операндов в памяти. Прямая и непосредственная адресация. Косвенная и косвенная регистровая, регистровая автоинкрементная и автодекрементная. индексная и базово-индексная адресация данных.

Способ адресации – это способ формирования исполнительного адреса по адресному коду команды.

Непосредственная адресация

При непосредственной адресации (НА) в адресном поле команды вместо адреса содержится непосредственно сам операнд. Этот способ применяется при выполнении арифметических операций, операций сравнения, для загрузки констант в регистры. Когда операндом является число, оно обычно представляется в дополнительном коде. При записи в регистр, имеющий разрядность, превышающую длину непосредственного операнда, операнд размещается в младшей части регистра, а оставшиеся свободными позиции заполняются значением знакового бита операнда. В адресном поле могут быть указаны только константы. Длина операнда редко превышает 16 бит. В подавляющем числе случаев 16 разрядов вполне достаточно, хотя для вычисления адресов const может быть длиннее. Основная проблема – не каждый операнд можно передать непосредственно – адресная часть команды как правило меньше машинного слова. 50-60% - до 8 бит, 70-80% - до 16 бит. Плюсы НА – малое время выполнения команды, экономия памяти. Недосток НА – размер непосредственного операнда ограничен длиной адресного поля команды, которое в большинстве случаев меньше длины машинного слова.

Непосредственная или литеральная	Add R4, #3	R4 <- R4+3 Для задания констант
-------------------------------------	------------	---------------------------------

Пример

Код операции	CA	Непосредственный операнд
--------------	----	--------------------------

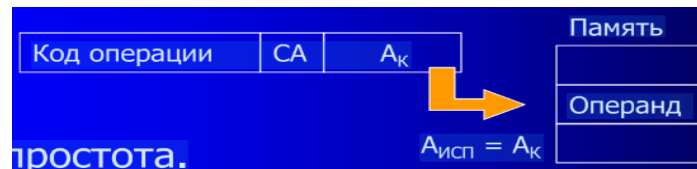
Прямая адресация

При прямой адресации (ПА) адресный код прямо указывает номер ячейки памяти операнда, к которой производится обращение, т.е. адресный код = исполнительный адрес. При всей простоте использования способ имеет существенный недостаток – ограниченный размер адресного пространства, так как для адресации к памяти большой емкости нужно «длинное» адресное поле. Однако главным недостатком ПА можно считать то, что адрес, указанный в команде, не может быть изменен в процессе вычислений (во всяком случае,

такое изменение не рекомендуется). Это ограничивает возможности по произвольному размещению программы в памяти (ОЗУ).

Прямая или абсолютная	Add R1,(1000)	R1 <- R1+M[1000] Иногда полезна для обращения к стат. данным
-----------------------	---------------	---

Пример

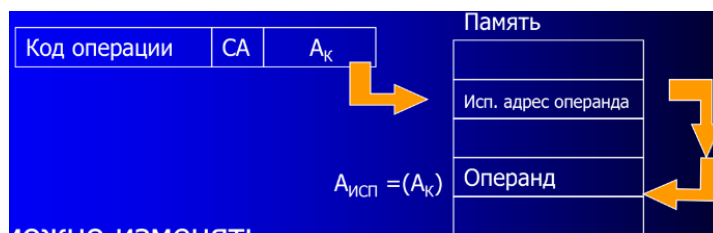


Косвенная адресация

Одним из путей преодоления проблем, свойственных ПА, может служить прием, когда с помощью ограниченного адресного поля команды указывается адрес ячейки, в свою очередь, содержащей полноразрядный адрес операнда. Этот способ известен как косвенная адресация (КА). При КА содержимое адресного поля команды остается неизменным, в то время как косвенный адрес в процессе выполнения программы можно изменять. Это позволяет проводить вычисления, когда адреса операндов заранее неизвестны и появляются лишь в процессе решения задачи. Дополнительно такой прием упрощает обработку массивов и списков, а также передачу параметров подпрограмм. Плюсы – можно изменять адреса операндов в процессе вычислений. Недостатком КА является необходимость в двукратном обращении к памяти: для извлечения адреса операнда и для обращения к операнду. Также задействуется лишняя ячейка памяти для хранения исполнительного адреса операнда. Многоуровневая или каскадная КА иногда удобна для обработки многомерных массивов.

Косвенная	Add R1,@(R3)	R1 <- R1+M[M[R3]] Если R3-адрес указателя p, то выбирается значение по этому указателю R1 <- R1+M[p]
-----------	--------------	--

Пример



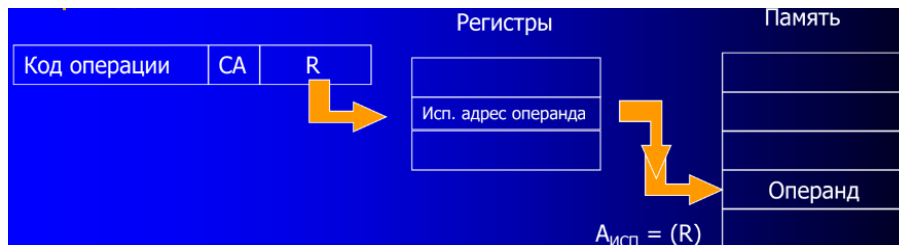
Косвенная регистровая адресация

Косвенная регистровая адресация (КРА) – КА, где исполнительный адрес операнда хранится не в ячейке основной памяти, а в регистре процессора. Соответственно, адресное поле команды указывает не на ячейку памяти, а на регистр. Достоинства – косвенный адрес в процессе выполнения программы можно изменять, упрощение обработки массивов и списков, а также передачи параметров подпрограмм. Плюсы – можно изменять адреса операндов в процессе вычислений. Также благодаря тому, что косвенный адрес хранится не в

памяти, а в регистре, для доступа к операнду требуется на одно обращение к памяти меньше.

Косвенная регистровая	Add R4,(R1)	$R4 \leftarrow R4 + M[R1]$ Для обращения по указателю или вычисленному адр.
-----------------------	-------------	--

Пример

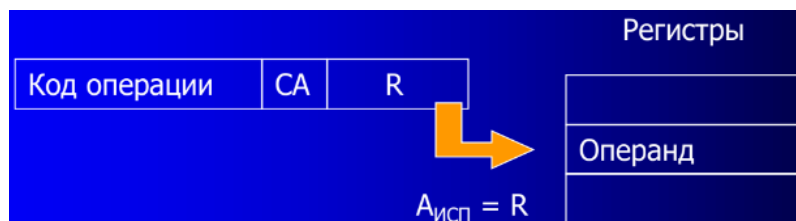


Регистровая адресация

Регистровая адресация (РА) напоминает ПА. При РА адресное поле инструкции указывает на регистр процессора. Обычно размер адресного поля в данном случае составляет 3-4 бита, что позволяет указать соответственно на один из 8 или 16 регистров общего назначения (РОН). Основные преимущества РА: короткое адресное поле (3-4 бита для 8-16 РОН) в команде и исключение обращений к памяти. Минусы – мало РОН.

Регистровая	Add R4,R3	$R4 \leftarrow R4 + R3$ Требуемое значение в регистре
-------------	-----------	---

Пример



Автоинкрементная и автодекрементная адресации

Автоиндексирование с увеличением содержимого индексного регистра носит название автоинкрементной адресации. Автоиндексирование с уменьшением содержимого индексного регистра носит название автодекрементной адресации. В большинстве ВМ увеличение или уменьшение содержимого индексного регистра до или после обращения к нему осуществляется автоматически как часть машинного цикла. Такой прием называется автоиндексированием.

Автоинкрементная	Add R1,(R2)+	$R1 \leftarrow R1 + M[R2]$ $R2 \leftarrow R2 + d$ Полезна для прохода в цикле по массиву с шагом: R2 - начало массива В каждом цикле R2 получает приращение d
Автодекрементная	Add R1,-(R2)	$R2 \leftarrow R2 - d$ $R1 \leftarrow R1 + M[R2]$ Аналогична предыдущей Обе могут использоваться для реализации стека

Пример

$A_{исп} = A_c + (R), R \leftarrow (R) + 1$	$R \leftarrow (R) + 1, A_{исп} = A_c + (R),$
Постинкрементное автоиндексирование	Преинкрементное автоиндексирование
$A_{исп} = A_c + (R), R \leftarrow (R) - 1$	$R \leftarrow (R) - 1, A_{исп} = A_c + (R),$
Постдекрементное автоиндексирование	Предекрементное автоиндексирование

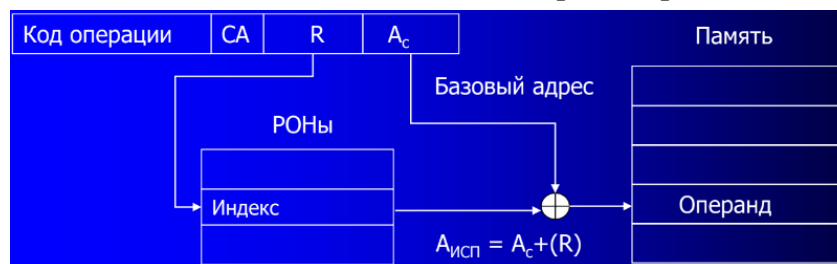
Индексная и базово-индексная адресация

При индексной адресации (ИА) поле A_c содержит адрес ячейки памяти, а регистр (явный/неявный) – смещение относительно этого адреса. Похож на базовую РА. Поскольку при ИА в поле указан полноразрядный адрес ячейки памяти, играющий роль базы, длина этого поля больше, чем при базовой РА.

Индексная	Add R3, (R1+R2)	R3 $\leftarrow$ R3+M[R1+R2] Иногда полезна при работе с массивами: R1 - база, R3 - индекс	Пример
-----------	-----------------	---	--------



Использование индексного регистра



Использование РОНов для хранения индекса (явный регистр)

Существует также ИА с масштабированием и смещением – содержимое индексного регистра умножается на коэффициент масштабирования и суммируется с A_c .

Индексная со смещением и масштабированием	Add R1, 100(R2)[R3]	R1 $\leftarrow$ R1+M[100]+R2+R3*d Для индексации массивов	Пример
---	---------------------	--	--------

30. Способы адресации операндов в памяти. Адресация операндов с использованием программного счетчика. Непосредственная и абсолютная, относительная и косвенная относительная адресация. (Ефимчик)

31. Способы адресации операндов в памяти. Стековая адресация памяти.

Стек представляет собой группу последовательно пронумерованных регистров (аппаратурный стек) или ячеек памяти, снабженных указателем стека

(обычно регистром) (УС), в котором автоматически при записи и считывании устанавливается номер (адрес) последней занятой ячейки стека (вершины стека). При операции записи заносимое в стек слово помещается в следующую по порядку свободную ячейку стека, а при считывании из стека извлекается последнее поступившее в него слово. Таким образом, в стеке реализуется правило «последний пришел - первый ушел».

Указанное правило при обращении к стеку реализуется автоматически, и поэтому при операциях со стеком возможно безадресное задание операнда — команда не содержит адреса ячейки стека, но содержит адрес (или он подразумевается) ячейки памяти или регистра, откуда слово передается в стек или куда помещается из стека.

При выполнении команды передачи в стек слова из регистра или ячейки ОП сначала указатель стека увеличивается на 1 (в перевернутом стеке уменьшается на 1), а затем слово помещается в ячейку стека, указываемую УС. При команде загрузки из стека регистра или ячейки памяти сначала слово извлекается из вершины стека, а затем указатель стека уменьшается на 1 (в перевернутом стеке увеличивается на 1). Как это ни кажется на первый взгляд удивительным, но при соответствующем расположении операндов в стеке можно вычислять выражения полностью безадресными командами, указывающими только вид операции. Такая команда извлекает из стека в соответствии с кодом операции один или два операнда, выполняет над ними предписанную операцию и заносит результат в стек.

Способ адресации – это способ формирования исполнительного адреса по адресному коду команды.

Используется в ВМ со стековой архитектурой.

Стековый доступ к памяти происходит в большинстве процессоров в следующих ситуациях:

- 1) При выполнении команд стекового доступа: "поместить в стек" push или "извлечь из стека" pop.
- 2) При выполнении команды вызова подпрограммы и при возврате из нее.
- 3) При входе в прерывание и возврате из него.

Положение стека в адресуемой памяти определяется содержимым указателя стека.

Содержимое указателя стека используется как адрес операнда-приемника при записи или как адрес операнда-источника при считывании из стека.

При обращении к стеку автоматически модифицируется и содержимое указателя стека, чтобы обеспечить следующее обращение к очередной ячейке стека.

Обращение к стеку - косвенно-регистровая адресация через указатель стека с автоиндексацией.

32. Команды различных форматов. Структура аппаратных средств для выполнения команд компьютера. (Филичева)

33. Форматы команд персональных компьютеров (на примере IBM PC/AT 8086). Команды арифметических и логических операций. Команды пересылки, передачи управления и обращения к подпрограммам. Команды ввода-вывода.(ЗЕВОК Ферзя)

Форматы команд: В команде для сведений о типе операции, адресной информации о нахождении обрабатываемых данных, а также для информации о месте хранения результатов выделяют поля. Каждую команду можно разбить на 2 части: операционную и адресную

Операционная часть	Адресная часть
--------------------	----------------

Типовая команда должна определять:

- подлежащую выполнению операцию;
- адреса исходных данных, над которыми выполняется операция;
- адрес, по которому должен быть помещён результат операции.

Выбор формата команд при создании ВМ влияет на многие характеристики будущей машины.

Необходимо принимать во внимание также следующие факторы :

- общее число различных команд;
- общую длину команды;
- тип полей команды (фиксированной или переменной длины) и их длина;
- простота декодирования;
- адресуемость и способы адресации;
- стоимость оборудования для декодирования и исполнения команды.

Длина команды

Влияет на организацию и ёмкость памяти, структуру шин, сложность и быстроедействие ЦП.

Чем длиннее команда - тем медленнее её выборка. Длина должна быть кратна разрядности шины данных.

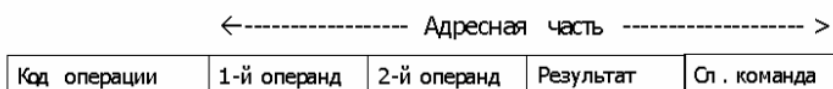
Различные способы адресации приводят к общему виду команды:

КОП – Код операции	СА – Способ адресации	Адресная часть
--------------------	-----------------------	----------------

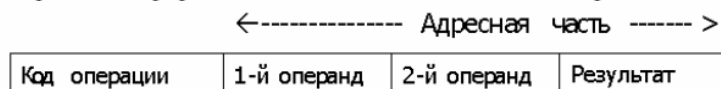
В большинстве ВМ одновременно используются несколько различных форматов команд.

Классификация по количеству адресов в команде:

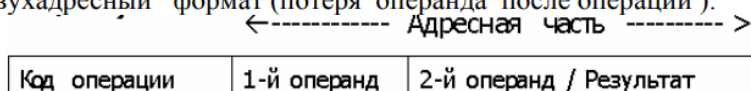
- Четырёхадресный формат:



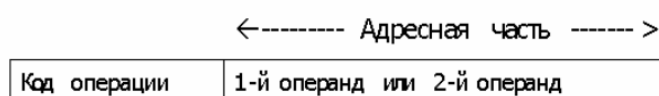
- Трёхадресный формат - добавляются команды переходов:



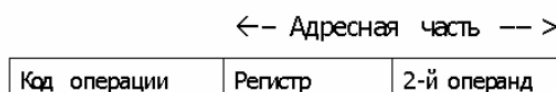
- Двухадресный формат (потеря операнда после операции):



- Одноадресный формат (аккумуляторная архитектура):



- Полуторадресный или регистровый формат:



Нульадресный формат (отдельные команды):



хз надо следующее или нет

Критерии выбора адресности команд:

- ёмкость запоминающего устройства;
- время выполнения программы;
- эффективность использования ячеек памяти при хранении программы.

С точки зрения увеличения ёмкости и эффективности использования памяти

- наиболее оптимальным вариантом будут одноадресные команды.

Суммарные потери времени для трёхадресной команды складываются из:

- времени на выборку команды;
- времени на выборку первого операнда;
- времени на выборку второго операнда;
- времени на запись результата в память.

Для одноадресной команды составляющие потерь по времени следующие:

- время на выборку команды;
- время на выборку операнда.

Определяющим при выборе является тип алгоритмов, на которые ориентирована ВМ. В последовательных программах - результат предыдущей операции используется для следующей, и выгодна одноадресная команда. В параллельных программах - результат пересылается в память, и трёхадресные команды будут эффективнее. В комбинированных программах лучше подойдут одноадресные и полуторадресные команды.

Оттранслированные команды записываются в соседние ячейки памяти (памяти программ, если память программ и данных разделены) в порядке их следования в программе. При естественном порядке выполнения команд в программе, адрес каждой следующей команды определяется по содержимому специального счетчика команд (IP или PC), который входит в состав процессора. Содержимое этого счетчика автоматически наращивается на 1 при выполнении очередной команды. При организации ветвления цикла или для перехода на подпрограмму в счетчик в счетчик команд принудительно записывается адрес перехода, указанный в ходе команды.

Команды арифметических и логических операций:

Команды арифметической и логической обработки обеспечивают арифметическую и логическую обработку информации в различных формах ее представления.

Для каждой формы представления чисел в архитектуре системы команд предусматривается определённый стандартный набор операций.

Помимо вычисления результата, формируются признаки (флаги) в АЛУ, характеризующие результат (например – Zero, Negative, Overflow, Carry).

Стандартный набор операций с целыми числами:

- двухместные (с двумя операндами) арифметические операции: «+», «-», «*», «/».

- одноместные арифметические операции. Например, абсолютное значение, изменение знака операнда.

- операции сравнения, устанавливающие признаки, характеризующие соотношения между величинами ($=$, $<$, $>$, $<=$, $>=$).

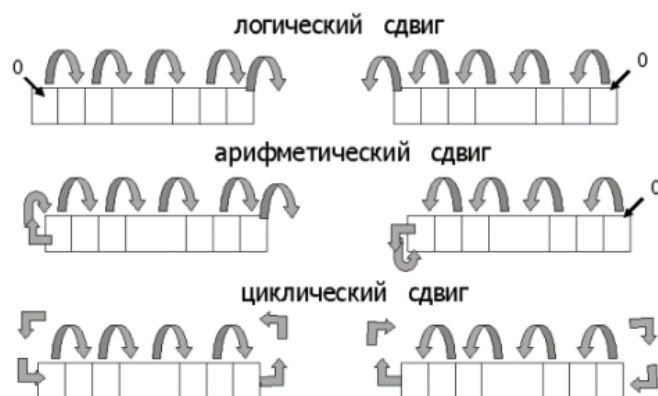
Часто дополняют - вычисление остатка от целочисленного деления, сложение с учётом переноса, вычитание с учётом займа, инкремент, декремент.

Типичный набор операций с числами в форме с плавающей запятой (ПЗ):

- основные арифметические операции: «+», «-», «*», «/».

- операции сравнения, устанавливающие признаки, характеризующие соотношения между величинами ($=$, $<$, $>$, $<=$, $>=$).

- операции преобразования: формы представления (между формой с фиксированной запятой (ФЗ) и плавающей запятой), формата представления (с одинарной и двойной точностью).



Логические команды предназначены для выполнения логических операций, как над отдельными битами слов, так и целыми байтами, словами и т.п.

Минимальный набор - «НЕ», «И», «ИЛИ », «сложение по модулю 2»

В дополнение практически во всех архитектурах системы команд (АСК) предусмотрены команды логического, арифметического и циклического сдвигов

Логический сдвиг – это сдвиг, при котором уходящий бит уходит, не влияя на оставшиеся биты, а на место появившегося бита записывается бит 0. Пример работы операции сдвига: число 10101010b дает число 01010100b, если сделать сдвиг влево на 1 бит, и число 01010101b, если сделать сдвиг вправо на 1 бит.

Арифметический сдвиг. При этом сдвиге слово рассматривается не просто как группа битов, а как целое число в дополнительном коде. При сдвиге влево ведет себя как логический сдвиг, при сдвиге вправо: уходящий бит уходит, не влияя на оставшиеся биты, а на место появившегося бита устанавливается бит, соответствующий знаку. Пример работы операции сдвига: число 11111010b = -6 дает число 11110100b = -12, если сделать сдвиг влево на 1 бит, и число 11111101b = -3, если сделать сдвиг вправо на 1 бит.

Легко заметить, что при арифметическом сдвиге сдвиг влево соответствует умножению на 2, а сдвиг вправо делению на 2.

Циклический сдвиг. При этом сдвиге уходящий бит появляется на месте появившегося. Пример работы операции сдвига: число 11111010b дает число 11110101b, если сделать сдвиг влево на 1 бит, и число 01111101b, если сделать сдвиг вправо на 1 бит.

Операции с десятичными числами:

Ранее широко использовались специальные форматы хранения - двоичнодесятичный код (BCD – Binary Coded Decimal).

Два формата: зонный и упакованный. Умножение и деление возможно только в зонном формате.

В ВМ первых поколений в АСК были специальные команды («+», «-», «*», «/») для десятичных чисел.

В АСК современных машин, соответствующие вычисления имитируются с помощью команд целочисленной арифметики.

Команды ввода вывода:

Могут быть разделены на 3 типа:

- Команды управления периферийными устройствами (ПУ).
- Команды проверки состояния ПУ.
- Команды ввода /вывода.

Начиная с этого момента беру инфу из левых источников, так что на свой страх и риск

Команды пересылки:

Для удобства практического применения и отражения их специфики команды данной группы удобнее рассматривать в соответствии с их функциональным назначением, согласно которому их можно разбить на следующие группы команд:

- 1) пересылки данных общего назначения;

- 2) ввода-вывода в порт;
- 3) работы с адресами и указателями;
- 4) преобразования данных;
- 5) работы со стеком.

Команды передачи управления:

Команды передач управления, или переходы, составляют очень важный класс в системе команд любого компьютера. Они служат для изменения нормальной последовательности выполнения команд или безусловно, или в результате анализа определенных условий. В микропроцессорах обычно бывает много команд такого типа. Типичным представителем можно считать команду «переход при ненулевом переносе» (JCN).

Поскольку это трехбайтовая команда, фаза выполнения может быть начата на третьем синхроимпульсе третьего машинного цикла. По этому синхроимпульсу БУС опрашивает состояние триггера переноса C. Если состояние C удовлетворяет заданному условию, т. е. если $C=1$, то, как показано на рис.4.10, содержимое регистра адреса данных передается на программный счетчик.

Таким образом, второй и третий байты команды становятся адресом команды, которая будет выбираться следующей. Если же, напротив, состояние триггера C не удовлетворяет условию, т. е. $C=0$, то программный счетчик сохраняет то значение, которое он получил в фазе выборки-дешифрации. Поэтому JCN не окажет влияния на порядок выборки команд.

Команды обращения к подпрограммам:

Хуй знает, что он хочет, я нашел только ассемблерные команды, возможно они и в прошлых пунктах нужны были:

1) Call label - команда переходит к метке label, сохраняя адрес возврата в стеке. Подпрограмма выполняется, управление возвращается к следующей инструкции.

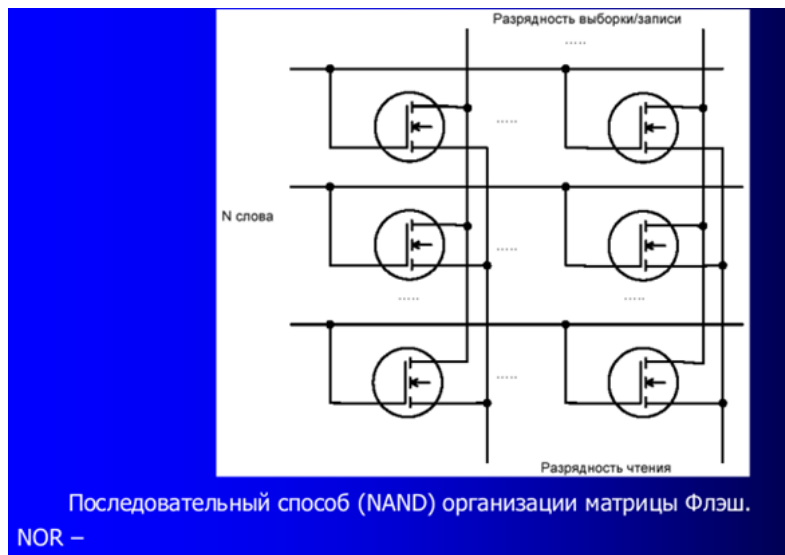
2) Ret - команда извлекает адрес возврата из стека и переходит к нему.

34. Flash память (Давидович).

FLASH - перепрограммируемая ПЗУ с электрическим стиранием. Стирание осуществляется сразу целой области ячеек : блока или всей микросхемы.

Флэш-память строится на одното́нзисторных элементах памяти (с “плавающим” затвором)

Различные технологии построения базовых элементов флэш - количество слоев, методы стирания и записи данных, структура. Наиболее широко известны NOR и NAND типы флэш-памяти, запоминающие транзисторы в которых подключены к разрядным шинам - параллельно и последовательно.



Минусы – относительно невысокая скорость передачи данных.

Элементы памяти флэш -ЗУ организованы в матрицы, разрядность данных для микросхем - 1-2 байта .

Операция чтения из флэш - как в обычных ЗУ с произвольным доступом (ОЗУ или кэш). Но запись – с некоторыми особенностями, аналогичными - ПЗУ. Перед записью данных в ячейки они должны быть очищены (стерты). Стирание - перевод элементов памяти в состояние единицы и возможно только сразу для целого блока ячеек (в первых микросхемах предусматривалось стирание только для всей матрицы сразу). Выборочное - невозможно.

Фактически при операции записи производится два действия: стирание и запись, но управление этими операциями производится внутренним автоматом и “прозрачно” для процессора.

Разбиение адресного пространства микросхемы флэш -памяти Sна блоки обычно двух видов: симметричное и асимметричное.

При симметричном (Flash File) - все блоки

